

Thesis for the Degree of Master of Computer Science (MCS)

**Comparative Benchmarking of CatBoost, TabPFN,
and TabFM for Explainable Multi-Tier Hair Fall
Risk Stratification**



Krishna Gautam

(LC ID: [LC ID])

Lincoln International College of Management & IT

Lincoln University College, Malaysia

September, 2026

Research Project for the Degree of Master of Computer Science
(MCS)

**Comparative Benchmarking of CatBoost, TabPFN,
and TabFM for Explainable Multi-Tier Hair Fall
Risk Stratification**

Supervised by Rabin Shrestha

A thesis submitted in partial fulfilment of the requirements for the
degree of Master of Computer Science (MCS)

Krishna Gautam

(LC ID: [LC ID])

Lincoln International College of Management & IT

Lincoln University College, Malaysia

September, 2026

Declaration

I hereby declare that this study entitled **Comparative Benchmarking of CatBoost, TabPFN, and TabFM for Explainable Multi-Tier Hair Fall Risk Stratification** is based on my original research work. Related works on the topic by other researchers have been duly acknowledged. I owe all the liabilities relating to the accuracy and authenticity of the data and any other information included hereunder.

.....

Signature

Name of the Student: Krishna Gautam

Registration Number: LC ID: [LC ID]

Date:

Recommendation

This is to certify that this thesis entitled **Comparative Benchmarking of CatBoost, TabPFN, and TabFM for Explainable Multi-Tier Hair Fall Risk Stratification**, prepared and submitted by **Krishna Gautam**, in partial fulfilment of the requirements of the degree of Master of Computer Science (MCS) awarded by Lincoln University College, Malaysia, has been completed under my supervision. I recommend the same for acceptance by Lincoln University College, Malaysia.

.....

Signature

Name of the Supervisor: Rabin Shrestha

Organization: Lincoln International College of Management & IT

Date:

Certificate

This thesis entitled **Comparative Benchmarking of CatBoost, TabPFN, and TabFM for Explainable Multi-Tier Hair Fall Risk Stratification** prepared and submitted by **Krishna Gautam** has been examined by us and is accepted for the award of the degree of Master of Computer Science (MCS) by Lincoln University College, Malaysia.

.....

Signature: Date signed:

External Examiner

Rabin Shrestha

Signature: Date signed:

Supervisor

.....

Signature: Date signed:

Head of Department / Principal

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Rabin Shrestha, for his guidance, suggestions and encouragement throughout this research. His comments at every stage helped me to shape the study and to improve this thesis.

I am thankful to Lincoln International College of Management & IT and to all the faculty members and staff of the Master of Computer Science programme for their teaching and support during my studies. I also thank the creators of the public datasets used in this study for making their data available, and the developers of the open-source software used in the experiments.

I am grateful to my classmates and friends for their discussions and help, and to my family for their patience, support and encouragement.

.....

Signature

Name of the Student: Krishna Gautam

Registration Number: LC ID: [LC ID]

Date:

List of Abbreviations/Acronyms

ALT	Alanine Aminotransferase
CUDA	Compute Unified Device Architecture
CV	Cross-Validation
EHR	Electronic Health Record
GPU	Graphics Processing Unit
JIT	Just-In-Time (compilation)
KNN	k-Nearest Neighbour
LUC	Lincoln University College
MAE	Mean Absolute Error
MCS	Master of Computer Science
NeurIPS	Conference on Neural Information Processing Systems
QWK	Quadratic Weighted Kappa
ROC-AUC	Receiver Operating Characteristic – Area Under the Curve
SHAP	Shapley Additive Explanations
SVM	Support Vector Machine
TabFM	Tabular Foundation Model
TabPFN	Tabular Prior-Fitted Network

Units and Conversions

%	Percent
g/dL	Grams per decilitre
GB	Gigabyte
mg/dL	Milligrams per decilitre
min	Minute
ng/mL	Nanograms per millilitre
s	Second
U/L	Units per litre
µg/dL	Micrograms per decilitre
µg/L	Micrograms per litre

ABSTRACT

Hair loss is influenced by many factors acting together, and no single measurement predicts it reliably. Earlier prediction studies used classical classifiers, reached conflicting conclusions about the best model, and did not test newer tabular foundation models. This study benchmarked a tuned gradient-boosted model, CatBoost, against two zero-shot tabular foundation models, TabPFN and TabFM, for classifying hair fall risk into three tiers (Low, Moderate and High) from structured clinical and lifestyle data, and explained the predictions of all three models with SHAP.

The dataset had 21,606 records with 20 predictors, assembled from two public datasets and adjusted using domain-informed relationships. A stratified 80/20 split gave 17,284 training records and 4,322 test records. CatBoost was tuned by grid search with five-fold cross-validation. TabPFN received the full training set as context without tuning, and TabFM received a 500-row context because its running time rose steeply with context size. The models were compared using accuracy, macro-averaged precision, recall and F1-score, ROC-AUC and McNemar's test, and were also retrained with 100, 500 and 2,000 training rows.

Accuracy was 78.51% for CatBoost, 78.57% for TabPFN and 77.88% for TabFM, and no pair of models differed significantly. With small training contexts both foundation models scored clearly higher than CatBoost: at 100 rows the accuracy was 60.53%, 71.54% and 74.41% respectively. All three models classified the Moderate tier least well. SHAP showed that iron, stress level, total protein, vitamin D, liver enzyme ALT, calcium and manganese were the most influential features in every model, with the same direction of effect, which reflects relationships built into the dataset rather than new clinical findings. TabFM was slow, and its full-context run could not be completed.

On this dataset, zero-shot foundation models matched a tuned gradient-boosted model in accuracy, depended much less on the amount of training data, and could be explained with SHAP. Because the results come from a constructed dataset and one data split, testing on real clinical data, a full-context evaluation of TabFM and repeated data splits are recommended.

Keywords: hair fall risk, tabular foundation models, CatBoost, TabPFN, TabFM, SHAP, explainable machine learning

Table of Contents

Declaration.....	ii
Recommendation.....	iii
Certificate.....	iv
Acknowledgements.....	v
Abbreviations/Acronyms.....	vi
Units and Conversions.....	vii
ABSTRACT.....	viii
Table of Contents.....	ix
List of Tables.....	xii
List of Figures.....	xiii
List of Appendices.....	xiv
CHAPTER 1 INTRODUCTION.....	1
1.1 Background.....	1
1.2 Statement of the problem.....	3
1.3 Research questions.....	4
1.4 Research objectives.....	5
1.5 Significance/Rationale of the study.....	5
CHAPTER 2 LITERATURE REVIEW.....	7
2.1 Machine learning for hair loss prediction.....	7
2.2 Tree ensembles and gradient boosting.....	9
2.3 Tree-based models and deep learning on tabular data.....	10
2.4 Transformers, attention and in-context learning.....	11
2.5 Tabular foundation models.....	11
2.6 Explainability with SHAP.....	13
2.7 Evaluating and comparing classifiers.....	14
2.8 Critical summary and research gap.....	15
CHAPTER 3 METHODOLOGY.....	17
3.1 Research design.....	17
3.2 Research approach.....	18
3.3 Study area.....	19
3.4 Study population.....	19
3.5 Sample selection.....	19

3.6 Sample size.....	19
3.7 Methods of data collection.....	20
3.8 Data analysis approach and tools.....	21
3.8.1 Data preprocessing.....	21
3.8.2 Descriptive analysis of the dataset.....	22
3.8.3 Models and justification.....	22
3.8.4 Experimental setup.....	23
3.8.5 Training context experiment.....	24
3.8.6 Explainability analysis.....	25
3.8.7 Evaluation metrics and statistical testing.....	25
3.8.8 Computational assessment.....	27
3.8.9 Tools and environment.....	27
3.9 Ethical considerations.....	28
3.10 Validity, reliability and reproducibility.....	28
CHAPTER 4 RESULTS AND DISCUSSION.....	29
4.1 Predictive performance of the three models.....	29
4.1.1 Overall performance.....	29
4.1.2 Per-class performance and confusion matrices.....	30
4.1.3 Statistical significance.....	34
4.2 Effect of training context size.....	35
4.3 Explainability of the predictions.....	36
4.3.1 Relationship between the features and the risk tier in the data.....	36
4.3.2 Global feature importance.....	37
4.3.3 Direction of the effects.....	41
4.3.4 Differences between the rankings.....	44
4.3.5 Patient-level explanations.....	44
4.4 Computational cost and implementation limitations.....	47
4.4.1 TabFM runtime.....	47
4.4.2 Software problems and workarounds.....	48
4.5 Discussion.....	49
4.5.1 Plausibility of the findings.....	49
4.5.2 Comparison with similar studies.....	50
4.5.3 Implications and cautions.....	51
4.5.4 Relation of the results to the research questions.....	52

CHAPTER 5 CONCLUSION AND RECOMMENDATIONS.....	53
5.1 Conclusion.....	53
5.2 Limitations of the study.....	54
5.3 Recommendations.....	55
REFERENCES.....	57
Appendix A: Source Code.....	60
Appendix B: Additional Figures.....	69
Appendix C: Result Tables From The Experiments.....	73
Appendix D: Dataset Description And Sample Records.....	74

List of Tables

Table 2.1	Summary of related studies on hair loss prediction.....	8
Table 2.2	Model families relevant to this study.....	15
Table 3.1	Link between the research objectives and the methods used.....	17
Table 3.2	Predictor variables.....	20
Table 3.3	Descriptive statistics of the continuous predictors (all 21,606 records).....	21
Table 3.4	Settings of the three models.....	24
Table 3.5	Experimental environment.....	27
Table 4.1	Performance of the three models on the test partition.....	29
Table 4.2	Precision, recall and F1-score by risk tier.....	30
Table 4.3	Share of records of each true tier assigned to a neighbouring tier.....	32
Table 4.4	Ordinal-aware measures.....	33
Table 4.5	Pairwise McNemar's test.....	34
Table 4.6	Accuracy with 95% Wilson score intervals.....	34
Table 4.7	Accuracy and macro-F1 with reduced training context.....	35
Table 4.8	Mean of the continuous predictors by risk tier.....	37
Table 4.9	Proportion of records with each binary flag, by risk tier.....	37
Table 4.10	Ten most important features in each model.....	40
Table 4.11	Approximate TabFM prediction time by context size.....	48
Table C.1	Full-context results (final_three_way_results.csv).....	73
Table C.2	McNemar's test (final_statistical_analysis.csv).....	73
Table C.3	Reduced-context results on the 2,000-row run (matched_2000ctx_results.csv; 500 test records).....	73
Table C.4	Per-class precision, recall and F1-score on the test partition.....	73
Table D.1	First three records of the dataset (full_name omitted; one column per record). 74	

List of Figures

Fig. 3.1	Block diagram of the research methodology.....	18
Fig. 4.1	CatBoost confusion matrix.....	30
Fig. 4.2	TabPFN confusion matrix.....	31
Fig. 4.3	TabFM confusion matrix.....	32
Fig. 4.4	Accuracy of the three models under reduced training context.....	35
Fig. 4.5	CatBoost global feature importance (SHAP).....	38
Fig. 4.6	TabPFN global feature importance (SHAP).....	39
Fig. 4.7	TabFM global feature importance (SHAP).....	40
Fig. 4.8	CatBoost SHAP beeswarm plot (High risk).....	41
Fig. 4.9	TabPFN SHAP beeswarm plot (High risk).....	42
Fig. 4.10	TabFM SHAP beeswarm plot (High risk).....	43
Fig. 4.11	CatBoost patient-level explanation.....	45
Fig. 4.12	TabPFN patient-level explanation.....	46
Fig. 4.13	TabFM patient-level explanation.....	47
Fig. B.1	CatBoost training curve.....	69
Fig. B.2	CatBoost confusion matrix with a 2,000-row training context (500 test records) 70	
Fig. B.3	TabPFN confusion matrix with a 2,000-row training context (500 test records)..71	
Fig. B.4	TabFM confusion matrix with a 2,000-row training context (500 test records)...72	
Fig. B.5	Accuracy of the three models with a 2,000-row training context.....72	

List of Appendices

Appendix A: Source Code.....	60
Appendix B: Additional Figures.....	69
Appendix C: Result Tables From The Experiments.....	73
Appendix D: Dataset Description And Sample Records.....	74

CHAPTER 1

INTRODUCTION

This chapter introduces the study. It presents the background of hair fall risk prediction (Section 1.1), the statement of the problem (Section 1.2), the research questions (Section 1.3), the research objectives (Section 1.4) and the significance of the study (Section 1.5). The thesis has five chapters. Chapter 2 reviews the literature, Chapter 3 describes the methodology, Chapter 4 presents and discusses the results, and Chapter 5 gives the conclusion and recommendations. The references and appendices follow Chapter 5.

Scope of the study. The study is limited to three-tier (Low, Moderate, High) hair fall risk classification from structured tabular clinical and lifestyle data. The dataset has 21,606 records, assembled from two public datasets and adjusted with domain-informed relationships. Three models are compared: CatBoost, TabPFN and TabFM. They are evaluated on one held-out test partition with accuracy, macro-averaged precision, recall and F1-score, ROC-AUC (area under the receiver operating characteristic curve) and McNemar's test, and their predictions are explained with SHAP (Shapley Additive Explanations).

Limitations of the study. The dataset was constructed and not drawn from a clinical registry, so the results show model capability on this data and are not a validated diagnostic tool. TabFM was evaluated with a 500-row training context because of its runtime, and the SHAP analysis of the two foundation models used a reduced training context and a small sample of test records. Scalp images, free-text records and other hair or scalp conditions are outside the scope of the study.

1.1 Background

Hair loss, known clinically as alopecia, affects people of all ages and backgrounds. Some daily shedding is a normal part of the hair growth cycle. A marked or sudden increase in shedding, however, often signals a deeper disturbance such as hormonal imbalance, nutrient deficiency, physical strain or prolonged psychological stress. Visible hair loss also carries an emotional cost, and it is commonly linked to lower self-confidence, anxiety and a reduced sense of well-being.

Hair grows in cycles. Each follicle passes through a growth phase, a short transition phase and a resting phase, after which the hair is shed and a new hair begins to grow. In a healthy

scalp only a small share of hairs is shed each day. Hair fall becomes a concern when more follicles than usual enter the resting phase and are shed together. This can be triggered by physical or psychological stress, inadequate nutrition, illness and hormonal change, so very different conditions can produce the same visible result. This is one reason why no single test identifies the risk, and why an estimate based on several measurements together is more informative than a judgement based on one.

Predicting who is at risk is difficult because no single biomarker reliably indicates hair loss. Risk arises from the combined and non-linear interaction of many factors, including iron and protein levels, vitamin D status, liver function, stress, sleep habits, exposure to chemical hair treatments, chronic illness and family history. Because these factors act together, it is impractical for a clinician or an individual to judge overall risk by observation alone. In practice, assessment still depends largely on self-reported history and delayed clinical examination, which are subjective, inconsistent between observers and hard to reach in under-resourced settings.

Much of the data that could support a more objective assessment is tabular. Laboratory results, demographic details, clinical history and lifestyle indicators are naturally stored as rows and columns, and Electronic Health Records (EHRs) hold such data in large volumes. Machine learning can find patterns in this kind of data that are hard to notice manually, and several studies have applied it to hair loss. Widowati et al. [1] built an explainable decision-support system for multi-class hair loss severity classification and showed that feature transformations must be restricted to within cross-validation folds to avoid inflated performance estimates. Sayyad and Midhunchakkaravarthy [2] developed a diagnostic model for Alopecia Areata, but it addressed a single condition rather than the wider set of risk factors behind hair loss. Sai et al. [3] compared individual classifiers with an ensemble method for hair fall prediction and found that the ensemble performed better on accuracy, precision and recall. Patel et al. [4] proposed a Random Forest framework using hereditary, hormonal, nutritional and medical-history factors. Chen et al. [5] combined genetic, hormonal, scalp and lifestyle data and observed that neural networks captured more complex feature interactions but were harder to interpret. Image-based work has also been done: Shakeel et al. [6] extracted texture, shape and colour features from scalp images and classified healthy hair and Alopecia Areata with Support Vector Machine (SVM) and k-Nearest Neighbour (KNN) classifiers.

On structured data, gradient-boosted decision trees have long been the standard approach. CatBoost [7] is one such method, designed to handle categorical features and to reduce target leakage through ordered target encoding. A newer family of models, tabular foundation models, takes a different route. These are transformers pretrained on large collections of synthetic datasets, and they predict directly from the training data supplied as context in a single forward pass, without dataset-specific training. TabPFN [8] introduced this idea, and TabFM [9], released by Google Research, extends it with attention applied alternately across rows and columns. Reports on the TabArena benchmark suggest that TabFM can outperform heavily tuned gradient-boosted models in a purely zero-shot setting [9], and an independent reproduction has confirmed its competitive accuracy while also documenting defects in the official implementation [10].

Predictive accuracy alone is not enough in a health setting, because a practitioner needs to know why a model reached its output. Both boosted ensembles and transformer models are opaque by default. SHAP (Shapley Additive Explanations) [11] addresses this by attributing a prediction to individual input features using Shapley values from cooperative game theory, and it can be applied to very different model types. This makes it suitable for comparing explanations across architectures.

The literature reviewed above shows that hair loss prediction has mostly relied on classical classifiers, that interpretability is a recognised concern for the more complex models, and that tabular foundation models have not been evaluated against gradient boosting on hair fall data. To our knowledge, no study has compared more than one foundation model on this task. These gaps motivate the present study, which benchmarks CatBoost, TabPFN and TabFM for multi-tier hair fall risk classification (Low, Moderate and High) on structured clinical and lifestyle data and applies SHAP to all three models.

1.2 Statement of the problem

Hair fall risk is hard to assess objectively. It depends on the combined effect of nutritional, hormonal, metabolic, lifestyle and hereditary factors, and no single indicator is reliable on its own (Section 1.1). Current practice relies on self-reported history and delayed clinical examination, which vary between observers and are difficult to access in under-resourced settings. Machine learning can model these interacting factors, but the existing work has three shortcomings that make it inadequate for this purpose.

First, the evidence on which model suits hair fall data is inconsistent. Sai et al. [3] found that an ensemble method outperformed individual classifiers, whereas Patel et al. [4] found that Random Forest outperformed XGBoost, CatBoost and LightGBM. Studies also differ in target (a single condition such as Alopecia Areata [2], image-based classification [6], or survey-based severity [1]), so their results cannot be compared directly. No clear, well-founded choice of model exists for structured hair fall data.

Second, tabular foundation models have not been tested on this problem. TabPFN [8] and TabFM [9] predict from supplied context without dataset-specific training, and TabFM has been reported to outperform heavily tuned gradient-boosted trees on the TabArena benchmark [9]. That evidence comes from general benchmarks, not from health survey data. It is therefore unknown whether these models can match a tuned gradient-boosted model such as CatBoost [7] on hair fall risk, or whether the result holds across more than one foundation model. The reliability of a newly released model such as TabFM also needs checking, since an independent reproduction found defects in its official implementation [10].

Third, the models that capture complex feature interactions are difficult to interpret. Chen et al. [5] observed that neural models gained accuracy at the cost of per-prediction interpretability, and the same opacity applies to boosted ensembles and transformers. Without feature-level explanations, a predicted risk level cannot be checked against known clinical reasoning, and this limits its usefulness for decision support.

These gaps make the research necessary. A comparison of CatBoost, TabPFN and TabFM on the same data, split and metrics, with SHAP explanations for all three, is needed to establish whether tuning-free foundation models can replace tuned gradient boosting for multi-tier (Low, Moderate, High) hair fall risk classification, and whether their predictions can be traced to interpretable clinical and lifestyle indicators.

1.3 Research questions

Based on the problem stated in Section 1.2, this study seeks to answer the following research questions:

1. Can the zero-shot tabular foundation models TabPFN and TabFM match the predictive performance of a tuned CatBoost model in classifying multi-tier (Low, Moderate, High) hair fall risk from structured clinical and lifestyle data, and are the differences between the three models statistically significant?

2. How does the amount of training data supplied (the training context size) affect the performance of CatBoost, TabPFN and TabFM, and which model uses limited data most efficiently?
3. Which clinical and lifestyle features contribute most to the predicted risk tier in each model, and are these contributions consistent across the three models and clinically plausible?
4. What are the practical computational costs and implementation limitations (runtime, memory and software reliability) of using each model, particularly the recently released TabFM?

1.4 Research objectives

General objective

To benchmark CatBoost, TabPFN and TabFM for explainable multi-tier (Low, Moderate, High) hair fall risk stratification from structured clinical and lifestyle data.

Specific objectives

1. To compare the predictive performance of a tuned CatBoost model with that of the zero-shot TabPFN and TabFM models using accuracy, macro-precision, macro-recall, macro-F1 and ROC-AUC, and to test whether the differences between them are statistically significant.
2. To evaluate how the training context size affects the performance of the three models, and to identify which model uses limited training data most efficiently.
3. To explain the predictions of the three models with SHAP and to compare the features that contribute most to the predicted risk tier across the models.
4. To assess the computational cost and implementation limitations of the three models, with particular attention to the recently released TabFM.

1.5 Significance/Rationale of the study

This study is justified by the gaps identified in Section 1.2. It compares a tuned gradient-boosted model with two tabular foundation models on the same hair fall data and explains all three with SHAP. The main beneficiaries and the ways they benefit are as follows.

Clinicians and dermatology practitioners. A model that estimates Low, Moderate or High hair fall risk from routine laboratory values and lifestyle information can support earlier

screening and a more consistent assessment than self-reported history alone. Because SHAP attributes each prediction to specific indicators such as iron, protein, vitamin D and stress level, a practitioner can check the result against clinical knowledge instead of accepting it as a black box. The study is intended as decision support, not as a replacement for clinical diagnosis.

Individuals at risk of hair loss. Risk tiers linked to identifiable, modifiable factors (for example nutritional status, stress and sleep habits) can help people recognise early warning signs and seek advice sooner. This is particularly useful where access to specialist dermatological care is limited.

Machine learning researchers and practitioners working with tabular data. The study gives a controlled comparison of two independently developed foundation models, TabPFN and TabFM, against a tuned CatBoost baseline. The context-size analysis shows how much training data each model needs, which matters where labelled clinical data is scarce. The record of TabFM's runtime behaviour and implementation problems helps others decide whether to adopt this recently released model.

Future researchers. The study uses one dataset, split, set of metrics and explanation method for all three models, and it reports the limits of the data and of the computing resources available. This provides a reproducible reference for later work, such as evaluation on real clinical data or ordinal classification methods.

Beyond these groups, the study has value for the research community as a documented example of how a new class of models can be evaluated fairly on a health problem. It shows the steps that a fair comparison needs: a tuned baseline, the same data and split for every model, a statistical test, a variation of the training size and an explanation method applied to every model. It also records the practical problems that arose with a recently released model, which are usually left out of benchmark reports.

Overall, the study helps establish whether tuning-free foundation models can replace tuned gradient boosting for structured health risk prediction while keeping predictions explainable.

CHAPTER 2

LITERATURE REVIEW

This chapter surveys published work relevant to the study. It first reviews machine learning studies on hair loss (Section 2.1). It then reviews the three families of methods used in this study: tree ensembles and gradient boosting (Section 2.2), the debate on trees versus deep learning for tabular data (Section 2.3), and transformer-based tabular foundation models (Sections 2.4 and 2.5). Section 2.6 reviews explainability with SHAP, and Section 2.7 reviews the measures and tests used to compare classifiers. Section 2.8 brings the literature together, criticises it and identifies the gap that this study addresses.

2.1 Machine learning for hair loss prediction

Machine learning has been applied to hair loss with structured survey data, clinical data and scalp images. Studies using structured data are the most relevant here because the present study also uses tabular clinical and lifestyle features.

Widowati et al. [1] developed an explainable decision-support system for multi-class hair loss severity classification. Their main methodological point was that feature transformations should be fitted only inside cross-validation folds, since fitting them on all the data inflates the estimated generalisation performance. This is a useful warning for any comparison of models, because leakage from preprocessing can favour one model over another without being noticed. Sayyad and Midhunchakkaravarthy [2] built a diagnostic model for Alopecia Areata and improved prediction for that condition, but their model did not cover the wider multi-factorial risk drivers of hair loss.

Several studies compared classifiers. Sai et al. [3] compared individual classification algorithms with an ensemble method for hair fall prediction and reported that the ensemble outperformed every individual algorithm on accuracy, precision and recall. Patel et al. [4] proposed a Random Forest framework using hereditary factors, hormonal markers, medical history and nutritional status, and found that Random Forest outperformed XGBoost, CatBoost and LightGBM on their dataset. Anderson and Roberts [12] analysed survey-based lifestyle and clinical indicators, including dandruff severity, chronic illness history, thyroid function and micronutrient status, and used ROC-AUC and macro-F1 to account for class imbalance. Chen et al. [5] combined genetic, hormonal and scalp-condition data with lifestyle variables and compared standard classifiers with neural networks. The neural

networks captured more complex feature interactions, but at the cost of interpretability of individual predictions.

Image-based approaches have also been reported. Shakeel et al. [6] extracted texture, shape and colour features from scalp images and classified healthy hair and Alopecia Areata with Support Vector Machine (SVM) and k-Nearest Neighbour (KNN) classifiers, obtaining 91.4% accuracy for SVM against 88.9% for KNN. Image-based work answers a different question from the present one, since it detects a visible condition, whereas risk stratification from laboratory and lifestyle data aims to estimate risk before hair loss is visible.

Table 2.1 summarises these studies.

Table 2.1: Summary of related studies on hair loss prediction

Ref.	Focus	Approach	Main finding	Limitation relevant to this study
[1]	Multi-class hair loss severity	Explainable decision-support system	Transformations must be kept inside cross-validation folds	Does not compare foundation models
[2]	Alopecia Areata	Machine learning diagnostic model	Improved prediction for one condition	Single condition, not multi-factorial risk
[3]	Hair fall prediction	Individual classifiers vs. an ensemble	Ensemble outperformed all individual classifiers	Classical models only
[4]	Hair fall prediction	Random Forest with hereditary, hormonal, medical and nutritional factors	Random Forest outperformed XGBoost, CatBoost and LightGBM	Conflicts with [3]; result specific to their dataset
[5]	Hair loss from genetic and scalp data	Classifiers vs. neural networks	Neural networks captured richer interactions	Lower interpretability
[6]	Healthy hair vs. Alopecia Areata	Image features with SVM and KNN	SVM 91.4%, KNN 88.9% accuracy	Image-based and binary; not tabular risk tiers
[12]	Hair loss severity from survey data	Survey-based clinical and lifestyle features	Used ROC-AUC and macro-F1 for class imbalance	Not compared with foundation models

A critical reading of these studies shows four points. First, the conclusions about the best model conflict: an ensemble was best in [3], whereas Random Forest beat several boosting methods in [4]. Because the datasets, targets and metrics differ, the two findings cannot be reconciled, and neither can be assumed to hold for a new dataset. Second, the studies target different outcomes (a single condition, image classes, or severity), so their reported

accuracies are not comparable with each other or with the present study. Third, methodological care varies. Only [1] stresses the control of leakage inside cross-validation, and only [12] explicitly uses metrics that account for class imbalance. Fourth, interpretability is treated unevenly: [5] found that more flexible models were harder to interpret, and among these studies only [1] was designed as an explainable system.

For the present study, these observations lead to three design decisions. The comparison of models is made on the same data, split and metrics, so that any difference can be attributed to the models. The metrics include macro-averaged measures, because the classes are unequal in size. The explanation method is applied to every model, so that interpretability is compared and not assumed.

2.2 Tree ensembles and gradient boosting

A single decision tree is easy to interpret but unstable, because small changes in the training data can change its structure. Ensemble learning combines several simple models to obtain a more accurate and stable prediction. In bagging-based ensembles such as Random Forest [13], many trees are grown on random samples of the data and random subsets of features, and their votes are combined. Random Forest is robust and needs little tuning, which is one reason it appears often in applied studies such as [4].

Boosting builds the ensemble in a different way: models are trained one after another, and each new model corrects the errors of the ensemble built so far. Friedman [14] showed that this can be treated as gradient descent in function space. At step m , a new tree h_m is fitted to the negative gradient of the loss (the pseudo-residuals) and added to the current model with a learning rate v :

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}} \quad (1)$$

$$F_m(x) = F_{m-1}(x) + v h_m(x) \quad (2)$$

Because each tree targets the remaining error, boosted trees can represent non-linear feature interactions that a single tree cannot, and they have long been the standard method for structured tabular data. XGBoost [15] made this approach scalable by adding a regularised objective, sparsity-aware split finding and efficient system design, and LightGBM [16] increased training speed with histogram-based splits and sampling techniques.

Encoding categorical variables is a recurring difficulty in boosting, since simple encodings can produce sparse features and leak information from the target. Prokhorenkova et al. [7] proposed CatBoost to address this. It computes target statistics for categorical values using an artificial ordering of the training data, so that the statistic for a record never uses that record's own label, and it applies the same idea to the boosting procedure itself (ordered boosting) to reduce prediction shift. CatBoost also builds symmetric (oblivious) trees, in which the same split condition is used across a whole level, which acts as a regulariser and makes prediction fast. The authors reported better results than other boosting implementations on the benchmarks they used. These properties make CatBoost a strong, well-established baseline for tabular clinical data, which is why it is used as the tuned reference model in this study. The finding of Patel et al. [4] that Random Forest outperformed CatBoost shows, however, that the ranking of tuned tree-based methods depends on the dataset.

2.3 Tree-based models and deep learning on tabular data

Deep learning has been very successful on images and text, but its advantage on tabular data has been questioned. Shwartz-Ziv and Armon [17] compared XGBoost with several recent deep models on datasets that were not used in the deep models' original papers. XGBoost generally performed better on those datasets and needed much less tuning, and an ensemble of the deep models with XGBoost performed better than either alone. Grinsztajn et al. [18] carried out a large benchmark and concluded that tree-based models remained state of the art on medium-sized tabular datasets, even when their speed advantage was ignored, and they discussed properties of tabular data (such as uninformative features and irregular target functions) that suit trees better than standard neural networks.

Tuning is a real cost. A gradient-boosted model has several hyperparameters, and finding good values means training the model many times, usually with cross-validation, and it needs some experience to choose sensible ranges. A method that gives comparable results without this step would save time, and it would also remove one source of unfairness in comparisons, namely that a baseline may be tuned more or less carefully than the new method. In the present study CatBoost was tuned by grid search so that the baseline was not handicapped.

These studies matter for the present work in two ways. They justify the use of a tuned gradient-boosted model as the strong baseline that any new method must be compared with. They also explain why tabular foundation models attracted attention: if a pretrained model can match tuned trees without a tuning stage, it would remove a step that currently takes time and expertise.

2.4 Transformers, attention and in-context learning

The transformer architecture [19] is built on the attention mechanism. Each element of the input is compared with every other element, and the comparison determines how much each element contributes to the representation of the others. In scaled dot-product attention, the queries Q , keys K and values V are combined as:

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3)$$

where d_k is the dimension of the keys. Attention lets the model relate any element of the input to any other regardless of their distance, and it can be computed for all elements in parallel, which is one reason why transformers can be trained on very large collections of data. Large language models showed that a transformer trained once on a very large collection of data can perform new tasks from examples placed in its input, without any change to its weights. This is called in-context learning, and a model that supports it across many tasks is called a foundation model.

Applied to tabular data, the idea is to pretrain a transformer on a very large number of artificial tabular datasets, so that it learns the general skill of predicting a target from labelled examples. At prediction time, the labelled training rows are given to the model as context together with the query rows, and the prediction is obtained in a single forward pass with no gradient-based training. The training set therefore does not change the model's weights, and it acts instead as input. A consequence is that the cost of a prediction grows with the size of the context, which matters in practice (Section 4.4).

2.5 Tabular foundation models

TabPFN [8] introduced this idea for tabular classification. It was presented as a transformer that solves small tabular classification problems in about a second, without hyperparameter tuning. A later version, published in Nature [20], extended the approach to larger and more varied tables. The authors reported that it outperformed tuned tree-based ensembles on

datasets of up to 10,000 samples and 500 features, in much less time. This is the evidence behind the expectation that zero-shot models can compete with tuned boosting on small and medium-sized tables. The dataset used in the present study has 21,606 records, so its full training partition (17,284 rows) is larger than the range for which that result was reported, which makes the comparison in this study informative about behaviour beyond that range.

TabFM [9], released by Google Research, is a more recent model. It alternates row-wise and column-wise attention over the input table before an in-context transformer produces the predictions, whereas TabPFN applies attention mainly across samples. On the TabArena benchmark suite it was reported to outperform heavily tuned gradient-boosted baselines in a purely zero-shot setting [9]. An independent evaluation by a third party [10], run on three machines and thirteen datasets, confirmed its competitive zero-shot accuracy against XGBoost, Random Forest and TabPFN, but it also documented four defects in the official implementation. The evidence reviewed here on TabFM comes from general benchmarks, not from health or dermatology data, and it comes from a company blog and a public repository, not from peer-reviewed publication. It should therefore be treated as provisional, which is one reason why testing TabFM independently on a new problem is useful.

The two foundation models differ in design, and the difference matters for how they are used. In TabPFN, attention is applied mainly across the samples of the table, so each query row is related to the labelled rows in the context. In TabFM, attention is applied alternately across rows and across columns before the in-context transformer reads the compressed representation of each row. Both approaches make the training data part of the input at prediction time. This has three consequences for a practical comparison. First, the amount of context that can be used is limited by memory and time, and a large training set may have to be reduced to a smaller context. Second, the cost of each prediction depends on the context size and not only on the model, which is unlike a boosted-tree model whose prediction cost is small once it is trained. Third, the training data must be available whenever predictions are made, which is a consideration for deployment. These points are examined for hair fall data in Chapter 4.

2.6 Explainability with SHAP

Boosted ensembles and attention-based models both give no direct explanation for an individual prediction. In clinical use this is a serious limitation, because a practitioner needs a reason for a risk estimate before acting on it. Shapley values, introduced in cooperative game theory [21], distribute the payoff of a game fairly among its players according to their contributions. Lundberg and Lee [11] applied this idea to prediction. In their SHAP framework each feature is a player and the model output is the payoff, and the prediction for an instance is split into additive contributions:

$$f(x) = \varphi_0 + \sum_{j=1}^M \varphi_j \quad (4)$$

where φ_0 is the expected model output and φ_j is the contribution of feature j . The Shapley value of feature j averages its marginal contribution over all subsets S of the other features F :

$$\varphi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F|-|S|-1)!}{|F|!} [f_{S \cup \{j\}}(x) - f_S(x)] \quad (5)$$

Lundberg and Lee [11] showed that this is the only additive attribution that satisfies the properties of local accuracy, missingness and consistency. Computing Eq. (5) exactly requires evaluating the model for a number of feature subsets that grows exponentially with the number of features, so practical algorithms are needed. For tree models, TreeExplainer [22] computes exact values in polynomial time by using the tree structure. For models without such structure, Kernel SHAP [11] estimates the values by querying the model on perturbed inputs and fitting a weighted linear regression. It works for any model, but the number of model calls grows with the number of explained instances and the number of samples per instance, which is costly when each call is expensive, as with an in-context transformer.

The results of a SHAP analysis are usually shown in three ways. A global bar plot ranks features by their mean absolute contribution. A beeswarm plot shows, for every explained record, the contribution of each feature and colours it by the feature's value, so that the direction of the effect can be read (for example, whether high values of a feature raise or lower the prediction). A waterfall plot shows how the contributions of the features of one record move the prediction from the baseline to its final value. All three are used in Chapter 4.

SHAP supports both a global view (mean absolute values give an importance ranking across the dataset) and a local view (the contribution of each feature to one patient's prediction). SHAP explains what a model has learned, and it does not show that the relationships are causal. The same method can be applied to different model types, which is why it was chosen here. Two cautions apply in practice. An explanation is only as reliable as the model, so a model that has learned a spurious pattern will give an explanation that reflects the pattern. In addition, Kernel SHAP values are estimates: they depend on the number of samples per instance and on the background data, and when only a few instances are explained the ranking of features with similar importance can change from one run to another. This is why the analysis in Chapter 4 relies on the shared set of leading features and not on the exact order.

2.7 Evaluating and comparing classifiers

Accuracy, the proportion of correct predictions, can be misleading when classes differ in size, because a model that favours the largest class can still score well. Sokolova and Lapalme [23] analysed performance measures for classification and described macro-averaging, in which a measure such as precision, recall or the F1-score is computed for each class and then averaged with equal weight, so that small classes count as much as large ones. For multi-class problems, ROC-AUC can be computed by treating each class in turn against all the others and averaging the results.

Reporting differences between two classifiers also requires a statistical test. McNemar's test [24] compares two classifiers evaluated on the same test records. It uses only the records on which the two classifiers disagree: b records that the first classified correctly and the second did not, and c records for the reverse. Under the null hypothesis that both make errors at the same rate, b and c are expected to be equal. Dietterich [25] compared several tests for supervised learning algorithms and found that McNemar's test has a low rate of false positive results and is a suitable choice when each algorithm can be trained and run only once, as was the case here. The test has limits. It does not measure variation from the choice of training set or from random seeds, and a non-significant result does not show that the classifiers are equivalent, only that no difference was detected.

Model selection needs care as well. Hyperparameters such as the depth of a tree should be chosen on data that is not used for the final test, and k -fold cross-validation does this by repeatedly training on part of the training data and scoring on the remaining part. Stratified

folds keep the class proportions in every fold, which matters when classes are unequal. Selecting hyperparameters on the test data would give an optimistic estimate of performance, which is the kind of leakage that Widowati et al. [1] warned about.

The risk tiers in this study are ordered (Low, Moderate, High), and standard multi-class classifiers treat them as unrelated labels. Ordinal classification methods take the order into account. Frank and Hall [26], for example, proposed a simple approach that turns an ordinal problem with k classes into $k-1$ binary problems, each asking whether the target is greater than a given tier, so that any binary classifier can be used. Such methods may improve results for the middle tier of an ordered scale. They were not part of this study, whose aim was to compare three methods on the standard multi-class task, and they are noted as further work (Section 5.3).

2.8 Critical summary and research gap

Table 2.2 brings the reviewed methods together.

Table 2.2: Model families relevant to this study

Model family	Representative studies	Strengths	Known limitations	Role in this study
Random Forest and other classical classifiers	[13], [3], [4]	Robust, little tuning	Ranking versus boosting varies by dataset	Not evaluated (context only)
Gradient-boosted trees	[14], [15], [16], [7]	Strong on tabular data; tuned baseline	Needs tuning; needs enough training data	CatBoost, tuned baseline
Neural networks	[5], [17], [18]	Can capture complex interactions	Often no better than trees on tabular data; harder to interpret	Not evaluated (context only)
Tabular foundation models	[8], [20], [9], [10]	No tuning; strong with small data	Cost grows with context; TabFM evidence provisional	TabPFN and TabFM, zero-shot
Post-hoc explanation	[21], [11], [22]	Model-agnostic feature attribution	Kernel estimation is costly for large-context models	SHAP for all three models

Three points follow from Table 2.2. Tuned gradient boosting is the method that any new tabular method must be compared with, so a fair benchmark needs a properly tuned CatBoost and not a default configuration. Tabular foundation models are promising mainly because they remove the tuning step and work with little data, so a fair benchmark also needs a varied training size and not only the full dataset. Finally, since both kinds of model are opaque, a fair benchmark for health data should compare what each model relies on,

and not only how accurate it is. The design of the present study follows these three points (Chapter 3).

A further point concerns the data. A dataset built by combining sources, as in this study, has feature-target relationships that are set by the rules used to build it. A model trained on such a dataset can be expected to recover those rules, and how well it does so is informative about the model. It says little about how the model would perform on patients from a real clinic, where the relationships are noisier and are not known in advance. For this reason the study is presented as a benchmark of model behaviour and not as a clinical validation.

The reviewed work shows the following.

1. Studies on hair loss prediction have used classical classifiers, ensembles, Random Forest, neural networks and image features [1]–[6], [12]. Their conclusions about the best model conflict ([3] versus [4]), and their targets differ, so the results are not comparable.
2. Interpretability remains a concern. The neural models in [5] were harder to interpret, and among the reviewed hair loss studies only [1] was designed as an explainable system.
3. Tuned gradient boosting is an established baseline for tabular data [17], [18], and CatBoost [7] is a leading implementation, but it has not been compared with tabular foundation models on hair fall data.
4. TabPFN [8], [20] and TabFM [9] have been evaluated only on general benchmarks. No reviewed study evaluates them on health survey data, compares the two models with each other and with a tuned gradient-boosted model, or applies a common explanation method to all of them. TabFM's reliability is also only partly established [10].
5. The behaviour of these models when training data is limited, which is common in health settings, has not been examined for hair fall risk, and neither has the cost of explaining large-context models with model-agnostic SHAP.

This study addresses these gaps by benchmarking CatBoost, TabPFN and TabFM on multi-tier hair fall risk classification with the same data, split and metrics, by varying the training context size, and by applying SHAP to all three models.

CHAPTER 3

METHODOLOGY

This chapter describes how the study was conducted to meet the research objectives stated in Section 1.4. It covers the research design and approach, the study area and population, sample selection and size, the data collection, the data analysis approach and tools, and the ethical considerations. Table 3.1 links each method to the objective it serves.

Table 3.1: Link between the research objectives and the methods used

Specific objective (Section 1.4)	Method used	Section
1. Compare predictive performance and test significance	Common train/test split, CatBoost tuning, TabPFN and TabFM inference, macro-averaged metrics, ROC-AUC, McNemar's test	3.5, 3.8.3, 3.8.4, 3.8.7
2. Evaluate the effect of training context size	Retraining all three models on 100, 500 and 2,000 training rows	3.8.5
3. Explain predictions with SHAP	Descriptive analysis of the data; TreeExplainer for CatBoost, KernelExplainer for TabPFN and TabFM	3.8.2, 3.8.6
4. Assess computational cost and implementation limitations	Recording runtimes and documenting implementation problems	3.8.8

3.1 Research design

The study used a quantitative, experimental design based on secondary data. It was a model-comparison (benchmarking) study: the same dataset, the same train/test partition and the same evaluation measures were applied to three models, so that differences in results could be attributed to the models and not to the data. No primary data were collected from people, so there was no field study area and no direct contact with respondents.

The study followed the pipeline in Fig. 3.1. The data were preprocessed and split once. CatBoost was tuned and trained on the training partition, while TabPFN and TabFM received the training rows as context without any training. All three models were evaluated on the same held-out test partition, compared with a statistical test, examined with reduced training contexts, and explained with SHAP.

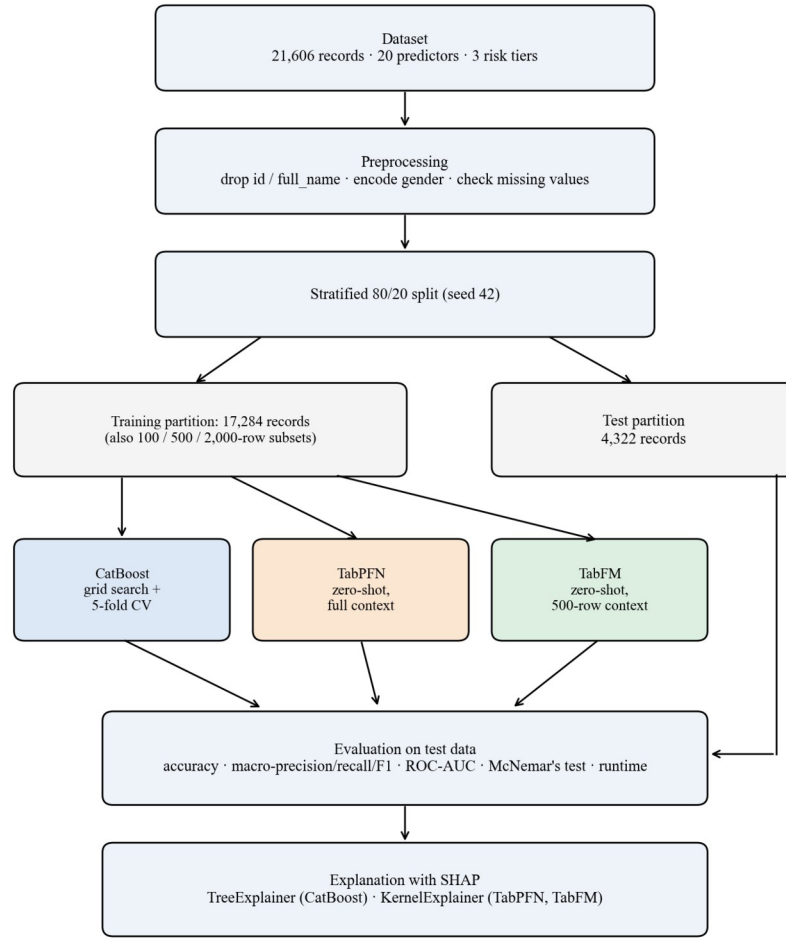


Fig. 3.1: Block diagram of the research methodology

The design was chosen because the research questions ask for a like-for-like comparison of models on one task. An experimental design is appropriate when the aim is to compare methods under controlled conditions, and a benchmarking study fits the objectives because none of them requires new data collection: each asks how existing methods behave on a defined task. A design based on a new prediction method would have answered a different question. A single fixed split and a fixed random seed (42) made the results reproducible. Each stage of Fig. 3.1 is linked to an objective in Table 3.1, so that no step was included without a purpose in the research.

3.2 Research approach

A quantitative, deductive approach was used. The proposition examined was that zero-shot tabular foundation models can match a tuned gradient-boosted model on multi-tier hair fall risk classification. It was examined with numerical performance measures and a statistical test, and the predictions were then explained with SHAP. A quantitative approach was

suitable because the research questions ask about measurable differences in performance, data efficiency and feature contribution.

3.3 Study area

The study was not a field study, so there was no geographical study area. The data were taken from two public datasets [27], [28], and all experiments were run in cloud notebook environments (Section 3.8.9).

3.4 Study population

The population was the set of patient records describing demographic, biochemical, clinical and lifestyle attributes related to hair fall. The dataset had 21,606 records with 23 columns: an identifier (id), a name (full_name), 20 predictor variables and the target variable hair_fall. There were no duplicate rows and no missing values. The target had three ordinal tiers: 0 (Low, 9,723 records, 45.0%), 1 (Moderate, 7,562 records, 35.0%) and 2 (High, 4,321 records, 20.0%). The classes were therefore unequal in size, which is the reason for the macro-averaged measures in Section 3.8.7.

3.5 Sample selection

The whole dataset was used, so no separate sampling of records was done. It was divided into a training and a test partition by a stratified 80/20 split with a fixed random seed of 42, so that class proportions were preserved in both parts. The test partition was held out and was not used for tuning CatBoost's hyperparameters. The same partition was used for all three models.

For the training-context experiment (Section 3.8.5), smaller training samples of 100, 500 and 2,000 rows were drawn at random (seed 42) from the training partition. TabFM's main evaluation used a 500-row context drawn in the same way.

3.6 Sample size

The full dataset had 21,606 records. The split gave 17,284 training records and 4,322 test records (1,945 Low, 1,513 Moderate and 864 High). CatBoost and TabPFN used all 17,284 training records, and TabFM used a 500-row context (about 2.9% of the training partition). The reduced-context runs used 100, 500 and 2,000 training rows. The 100-row and 500-row runs were evaluated on all 4,322 test records, and the 2,000-row run on a fixed random subset of 500 test records.

3.7 Methods of data collection

No primary data were collected from people. The study used secondary data and computational modelling and measurement.

The dataset was assembled from two public sources. The physiological biomarker data came from the Kaggle Hair Loss Dataset [27], chosen for its continuous clinical indicators relevant to hair health. The demographic and lifestyle indicators came from the Mendeley Hair Fall Problem Survey Dataset [28], chosen for its binary lifestyle and hereditary risk factors. Respondent names were removed before processing. Because the two sources came from different respondent groups, feature values were adjusted using relationships between risk factors and hair loss reported in the dermatological literature, and this produced the final dataset. It was therefore a constructed dataset and not a clinical registry. The consequences for the interpretation of the results are discussed in Chapter 4 and Chapter 5.

The 20 predictor variables fell into four groups (Table 3.2).

Table 3.2: Predictor variables

Group	Variables
Demographics and heredity	age (years); gender (Female, Male, Other); family_hair_fall_history (0/1)
Serum and physiological biomarkers	total_protein (g/dL); iron (µg/dL); calcium (mg/dL); vitamin_d (ng/mL); manganese (µg/L); alt_liver (U/L); body_water_content (%); stress_level (0–40, modelled on the Perceived Stress Scale)
Hair condition scores	total_keratine and hair_texture (composite 0–100 scores, not standard laboratory measurements)
Clinical and lifestyle flags (0/1)	chronic_illness, anemia, stress, late_night_sleep, sleep_disturbance, water_reason (exposure to hard or chemically treated water), chemical_use (hair dyes, relaxers and other chemical treatments)

Table 3.3 gives the range of the continuous predictors in the full dataset.

Table 3.3: Descriptive statistics of the continuous predictors (all 21,606 records)

Predictor	Mean	Standard deviation	Minimum	Maximum
age (years)	32.83	9.67	20.0	55.0
total_protein (g/dL)	7.20	0.60	4.8	9.5
calcium (mg/dL)	9.30	0.60	7.0	11.5
iron (µg/dL)	98.47	30.94	10.0	217.0
vitamin_d (ng/mL)	28.11	11.75	4.0	78.4
alt_liver (U/L)	25.66	13.78	5.0	81.0
manganese (µg/L)	8.02	2.98	1.0	20.0
body_water_content (%)	54.98	7.89	35.0	75.0
stress_level (0–40)	18.03	7.90	0.0	40.0
total_keratine (0–100)	49.88	19.80	0.0	100.0
hair_texture (0–100)	50.16	19.67	0.0	100.0

The predictors cover plausible ranges for adults aged 20 to 55 years. The two hair-condition scores span the whole 0 to 100 scale with means near 50, and the stress score spans the whole 0 to 40 scale. The gender attribute had 12,109 male, 9,485 female and 12 other records, so the "other" group was too small to be analysed separately.

Model outputs (predictions, class probabilities, SHAP values) and running times were collected by running the experiments in Section 3.8, which is the measurement part of the study.

3.8 Data analysis approach and tools

The data were analysed in the steps below. The analysis was done in Python, and the tools are listed in Section 3.8.9. The code is given in Appendix A.

3.8.1 Data preprocessing

The following steps were applied before modelling.

1. Identifier removal. The columns `id` and `full_name` carry no predictive information and could create false associations, so they were removed.
2. Encoding. The gender attribute was mapped to numbers (0 = Female, 1 = Male, 2 = Other). The target was already coded as 0, 1 and 2, so it needed no change. All other predictors were numeric or binary and were used as they were. TabPFN and TabFM received these raw values without scaling.

3. Missing values. The data were checked and no missing values were found, so no imputation was needed.

No fitted transformation (such as scaling or target encoding) was applied, so the warning of Widowati et al. [1] about leakage from preprocessing did not apply to the modelling stage: nothing was learned from the test records before evaluation.

3.8.2 Descriptive analysis of the dataset

Before modelling, the relationship between each predictor and the risk tier was described in the full dataset. For each continuous predictor, the mean was calculated in each risk tier. For all predictors, Spearman's rank correlation with the ordinal risk tier was calculated. This analysis was done with the pandas and SciPy packages. It was not used to select features or to train any model. Its purpose was to provide an independent reference against which the SHAP explanations could be compared (Section 4.3).

3.8.3 Models and justification

Three models were selected to represent three different approaches to tabular prediction, in line with Objective 1.

3.8.3.1 CatBoost

CatBoost [7] is a gradient-boosted decision tree method that builds trees one after another, each correcting the errors of the ensemble so far, as described in Section 2.2. At iteration t it minimises a regularised objective:

$$L^{(t)} = \sum_i L(y_i, F_{t-1}(x_i) + h_t(x_i)) + \Omega(h_t) \quad (6)$$

where L is the multi-class cross-entropy loss, F_{t-1} is the ensemble built so far, h_t is the new tree and Ω penalises tree complexity. CatBoost reduces target leakage through ordered target statistics and builds symmetric (oblivious) trees, which limits overfitting. It was selected as the tuned, well-established baseline whose performance the foundation models had to match. Grid search was used to tune two hyperparameters that control model complexity and learning speed: the tree depth (deeper trees model more interactions but overfit more easily) and the learning rate (smaller values need more trees).

3.8.3.2 TabPFN

TabPFN [8], [20] is a transformer pretrained offline on synthetic datasets. It does not train on the target data. It receives the labelled training rows D_{train} and a query x_{test} together and returns the predictive distribution in a single forward pass:

$$P(y_{\text{test}}/x_{\text{test}}, D_{\text{train}}) = \int P(y_{\text{test}}/x_{\text{test}}, \theta) P(\theta / D_{\text{train}}) d\theta \quad (7)$$

This is done with the self-attention mechanism of Eq. (3). TabPFN was selected as the established tabular foundation model, which needs no hyperparameter tuning or feature scaling.

3.8.3.3 TabFM

TabFM [9] is a tabular foundation model released by Google Research. It alternates row-wise and column-wise attention over the table before an in-context transformer produces the prediction, and it also needs no training or tuning:

$$P(y_{\text{test}}/x_{\text{test}}, D_{\text{train}}) = \text{TabFM}(x_{\text{test}}, D_{\text{train}}) \quad (8)$$

TabFM was selected as a second, independently developed foundation model, so that the study could test whether the behaviour of zero-shot models holds for more than one architecture.

3.8.4 Experimental setup

CatBoost. A grid search was run over tree depth $\{4, 6, 8\}$ and learning rate $\{0.01, 0.03, 0.1\}$. Each combination was scored by stratified 5-fold cross-validation on the training partition, using macro-F1. All models used the MultiClass loss, `l2_leaf_reg` = 3.0, at most 1,000 iterations, early stopping after 50 rounds and a random seed of 42. The best combination was depth = 4 and learning rate = 0.1 (cross-validation macro-F1 = 0.7665). The final model was trained on the full training partition with this setting. In this final fit, early stopping was monitored on the test partition, so the test data influenced the number of trees selected. This is noted as a limitation in Section 5.2.

TabPFN. The default configuration of the TabPFN package was run on a GPU with a random seed of 42. The full training partition (17,284 rows) was supplied as context, and no tuning was done.

TabFM. The PyTorch implementation of TabFM version 1.0.0 was used with a single ensemble member (`n_estimators` = 1). A context of 500 rows was drawn at random (seed

42) from the training partition, because inference time grew steeply with context size and a full-context evaluation could not be completed within the limits of the cloud platforms (Section 4.4). The asymmetry between TabFM's context and that of the other two models is the reason for the experiment in Section 3.8.5.

The settings of the three models are summarised in Table 3.4.

Table 3.4: Settings of the three models

Setting	CatBoost	TabPFN	TabFM
Tuning	Grid search, 5-fold stratified cross-validation, macro-F1	None	None
Training data used	17,284 rows	17,284 rows (context)	500 rows (context)
Main parameters	depth 4, learning rate 0.1, l2_leaf_reg 3.0, up to 1,000 iterations, early stopping 50 rounds, MultiClass loss	Default configuration, GPU	Version 1.0.0, PyTorch backend, n_estimators = 1
Random seed	42	42	42 (context sampling)

The choices in the setup have reasons. The split was stratified so that the three tiers had the same proportions in the training and test partitions. Macro-F1 was used to select the CatBoost setting because it gives each tier equal weight, which is consistent with the macro-averaged measures used for evaluation. Five folds were used as a compromise between the reliability of the score and the computing time. Early stopping ends training when the score has not improved for 50 rounds, which limits overfitting. The grid was small (nine combinations) because it covered the two parameters that most affect complexity and speed and kept the search within the available computing time.

The tuning effort was deliberately unequal: CatBoost was tuned by grid search, while TabPFN and TabFM were not tuned. This asymmetry is central to the research question, which asks whether models that need no tuning can match one that does.

3.8.5 Training context experiment

To meet Objective 2, all three models were retrained with reduced training contexts of 100, 500 and 2,000 rows, drawn at random (seed 42) from the training partition. CatBoost was trained on the sampled rows with the configuration above, and TabPFN and TabFM received the sampled rows as context. The 100-row and 500-row runs were evaluated on the full test partition. Because TabFM inference was slow, the 2,000-row run was evaluated on a fixed random subset of 500 test records, for all three models. Results at 2,000 rows are therefore not directly comparable with the other sizes.

3.8.6 Explainability analysis

To meet Objective 3, post-hoc explanations were produced with SHAP [11], whose additive form is given in Eq. (4) and whose Shapley values are defined in Eq. (5).

For CatBoost, TreeExplainer [22] was used, which computes exact Shapley values from the tree structure, on the full test partition. TabPFN and TabFM have no tree structure, so KernelExplainer was used, which estimates the values by querying the model repeatedly. PermutationExplainer, which had been planned, failed because of a numba error in the cloud environments (Section 4.4). Because each query passes through the full in-context mechanism, the analyses of the two foundation models used a reduced training context, a background sample of 15 records, 100 kernel evaluations per explained record, and a random subset of the test partition (30 records for TabPFN and 20 for TabFM). The High risk class was chosen for the beeswarm and waterfall plots because it is the tier of greatest clinical interest and because a single class is needed to show the direction of an effect. Global importance plots, beeswarm plots and patient-level waterfall plots were produced, and the beeswarm and waterfall plots were drawn for the High risk class. Features were compared across models by rank and not by the size of the values, since the sample sizes differed.

3.8.7 Evaluation metrics and statistical testing

To meet Objective 1, predictions were evaluated on the test partition with macro-averaged measures [23], which give the three classes equal weight. TP_k , FP_k and FN_k are the true positives, false positives and false negatives of class k , and N is the number of test records.

$$Accuracy = \frac{\sum_{k=0}^2 TP_k}{N} \quad (9)$$

$$Macro - Precision = \frac{1}{3} \sum_{k=0}^2 \frac{TP_k}{TP_k + FP_k} \quad (10)$$

$$Macro - Recall = \frac{1}{3} \sum_{k=0}^2 \frac{TP_k}{TP_k + FN_k} \quad (11)$$

$$Macro - F1 = \frac{1}{3} \sum_{k=0}^2 \frac{2 \cdot Precision_k \cdot Recall_k}{Precision_k + Recall_k} \quad (12)$$

Because the tiers are ordered, four further measures were calculated from the confusion matrices. Cohen's kappa [29] measures the agreement between the predicted and true tiers beyond the agreement expected by chance:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \quad (13)$$

where p_o is the observed share of correct predictions and p_e is the share expected by chance from the row and column totals of the confusion matrix. Quadratic weighted kappa [30] uses the same idea but penalises an error by the squared distance between the true and predicted tiers, so that confusing Low with High counts four times as much as confusing neighbouring tiers. The share of predictions within one tier of the true tier and the mean absolute error, in tiers,

$$MAE = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i| \quad (14)$$

describe how far the errors were from the true tier.

A confusion matrix counts, for each true tier, how many records were assigned to each predicted tier. Precision for a tier is the share of records predicted as that tier that truly belong to it, and recall is the share of records of that tier that were found. The F1-score combines the two into one value that is high only when both are high. Accuracy is the share of all records that were classified correctly.

ROC-AUC was computed with a one-vs-rest strategy and macro-averaged. Per-class F1-scores and confusion matrices were also produced. Macro-averaging was used because the classes were unequal in size (Section 3.4).

Pairwise differences in accuracy between the models were tested with McNemar's test [24] with continuity correction on the same test records, using the counts b and c of records that one model classified correctly and the other did not:

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c} \quad (15)$$

The uncertainty of each accuracy value that comes from the finite size of the test sample was described with 95% Wilson score confidence intervals [31], calculated from the accuracy and the number of test records. A difference was called significant at $\alpha = 0.05$. McNemar's test was chosen because each model was trained and run once on one split, a

situation for which it has a low false-positive rate [25]. The test was applied to the full-context evaluation only.

3.8.8 Computational assessment

To meet Objective 4, the running time of each model was recorded during training or context fitting and during prediction, and the practical problems met when running the models (runtime growth, software errors and the workarounds used) were documented. Running times were measured as wall-clock time with the notebook's timer around the training or context-fitting call and around the prediction call. For TabFM, predictions for the full test partition were made in batches of 50 records, with the results written to a checkpoint file after each batch, so that a run interrupted by the session limit of the cloud platform could be resumed. Because the runs were made on shared cloud hardware, the times are approximate and can vary between sessions, and they are used to show how the cost changes with the context size and not to give exact benchmarks. The software problems were documented as they occurred, together with the diagnostic steps and the workaround used for each, so that they can be reported in Section 4.4.

3.8.9 Tools and environment

The experiments were written in Python in cloud notebooks on Kaggle and Google Colaboratory, using an NVIDIA T4 GPU (Table 3.5). Data handling used pandas and NumPy, and scikit-learn [32] was used for the split, cross-validation and metrics. The models came from the catboost, tabpfn and tabfm (PyTorch backend) packages, the explanations from the shap package, the significance test from statsmodels, and the plots from matplotlib and seaborn.

Table 3.5: Experimental environment

Item	Specification
Platform	Google Colaboratory and Kaggle Notebooks (cloud)
GPU	NVIDIA T4 (Colab: one T4; Kaggle: two T4, one used)
Memory	About 13 GB (Colab) and 30 GB (Kaggle)
Operating system	Ubuntu 22.04
Language and framework	Python [version to be confirmed]; PyTorch with CUDA
Main libraries	CatBoost, TabPFN, TabFM 1.0.0, SHAP, scikit-learn, statsmodels, pandas, NumPy

3.9 Ethical considerations

The study used only secondary data from two public datasets [27], [28]. No participants were recruited, no interviews or surveys were carried out, and no intervention was made, so ethical approval for data collection was not required. Respondent names were removed before processing, the columns `id` and `full_name` were dropped before modelling, and no attempt was made to identify any individual. The dataset was adjusted with domain-informed relationships and is therefore a constructed dataset. It is described as such throughout the thesis, and the results are presented as a benchmark of model behaviour. They are not presented as a clinical finding or as a diagnostic tool. Model outputs must not be used to make decisions about real patients without validation on real clinical data and review by a qualified clinician.

3.10 Validity, reliability and reproducibility

Several steps were taken to make the comparison valid and the results reproducible. To protect the validity of the comparison, all three models used the same records, the same split and the same measures. The test partition was not used for hyperparameter tuning (with the exception of the early stopping monitor noted in Section 3.8.4), and no fitted transformation was applied to the data, so information from the test records did not flow into the models through preprocessing.

To protect reliability, random seeds were fixed at 42 wherever a random choice was made in the modelling: the split, the cross-validation folds, the training contexts and the model initialisation. The result of a single seed and a single split cannot show how much the numbers vary from run to run, and this is acknowledged as a limitation in Section 5.2.

To support reproducibility, the code is given in Appendix A, the software packages are listed in Section 3.8.9, and the dataset is described in Section 3.7. Some parts cannot be reproduced exactly. The random samples of test records used for the SHAP analyses were not fixed by a seed when the experiments were run, so a repeated run would select different records. TabPFN is used through a package that needs a licence token, and TabFM version 1.0.0 may change in later releases.

Because the dataset was constructed, the study has no external validity for real patients until it is repeated on clinical data. Its internal validity, meaning whether the comparison between the models is fair, rests on the steps described above.

CHAPTER 4

RESULTS AND DISCUSSION

This chapter presents the findings of the study in the order of the specific objectives in Section 1.4. Section 4.1 compares the predictive performance of the three models (Objective 1). Section 4.2 reports the effect of training context size (Objective 2). Section 4.3 presents the descriptive analysis of the data and the SHAP explanations (Objective 3). Section 4.4 reports computational cost and implementation limitations (Objective 4). Section 4.5 discusses the plausibility of the findings and compares them with similar studies. Unless stated otherwise, all results were obtained on the held-out test partition of 4,322 records (1,945 Low, 1,513 Moderate and 864 High), and CatBoost and TabPFN used all 17,284 training records while TabFM used a 500-row context.

4.1 Predictive performance of the three models

4.1.1 Overall performance

Table 4.1 gives the overall results. All three models scored far above the accuracy of 33.3% expected from random guessing on three classes, and above the 45.0% obtained by always predicting the largest class (Low). This shows that the features carry real information about hair fall risk. TabPFN had the highest accuracy (78.57%), followed by CatBoost (78.51%) and TabFM (77.88%). TabPFN was highest on macro-recall, macro-F1 and ROC-AUC, and CatBoost was highest on macro-precision. The largest gap between any two models was 0.0072 on macro-F1 and 0.0052 on ROC-AUC. TabPFN achieved this without any tuning, whereas CatBoost needed a grid search, and TabFM came within 0.69 percentage points of TabPFN using under 3% of the training records.

Table 4.1: Performance of the three models on the test partition

Model	Training context	Accuracy	Macro-precision	Macro-recall	Macro-F1	ROC-AUC
CatBoost	17,284	78.51%	0.7915	0.7662	0.7763	0.9189
TabPFN	17,284	78.57%	0.7875	0.7722	0.7787	0.9232
TabFM	500	77.88%	0.7904	0.7608	0.7715	0.9180

The ROC-AUC values of about 0.92 are the average, over the three one-vs-rest comparisons, of the probability that a randomly chosen record of one tier receives a higher score for that tier than a randomly chosen record of the other tiers. This ranking quality is high and is very similar for the three models, which agrees with the close accuracy values.

4.1.2 Per-class performance and confusion matrices

Table 4.2 shows precision, recall and F1-score for each risk tier. All three models classified the Low and High tiers better than the Moderate tier in terms of F1-score, and the pattern was the same for every model.

Table 4.2: Precision, recall and F1-score by risk tier

Model	Tier	Precision	Recall	F1-score
CatBoost	Low	0.85	0.86	0.86
CatBoost	Moderate	0.68	0.74	0.71
CatBoost	High	0.84	0.70	0.77
TabPFN	Low	0.86	0.85	0.86
TabPFN	Moderate	0.68	0.73	0.71
TabPFN	High	0.82	0.73	0.78
TabFM	Low	0.87	0.83	0.85
TabFM	Moderate	0.66	0.76	0.71
TabFM	High	0.85	0.69	0.76

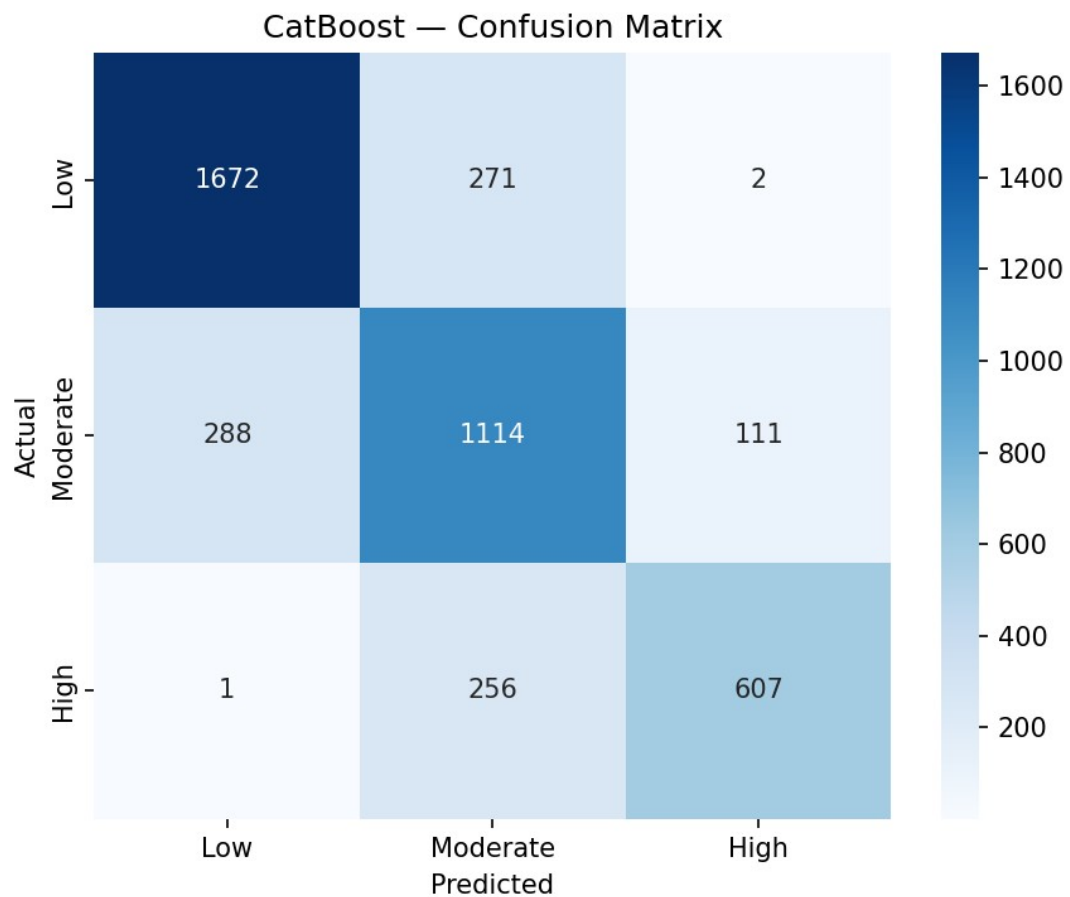


Fig. 4.1: CatBoost confusion matrix

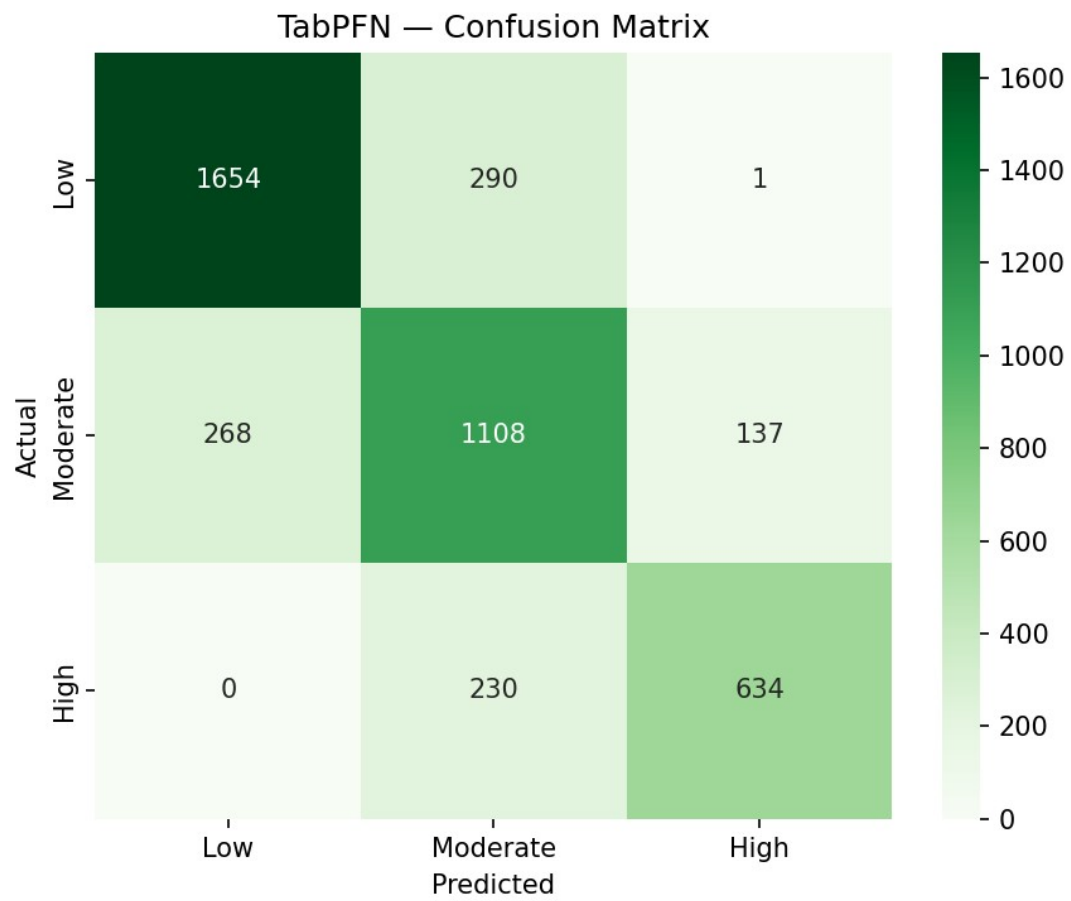


Fig. 4.2: TabPFN confusion matrix

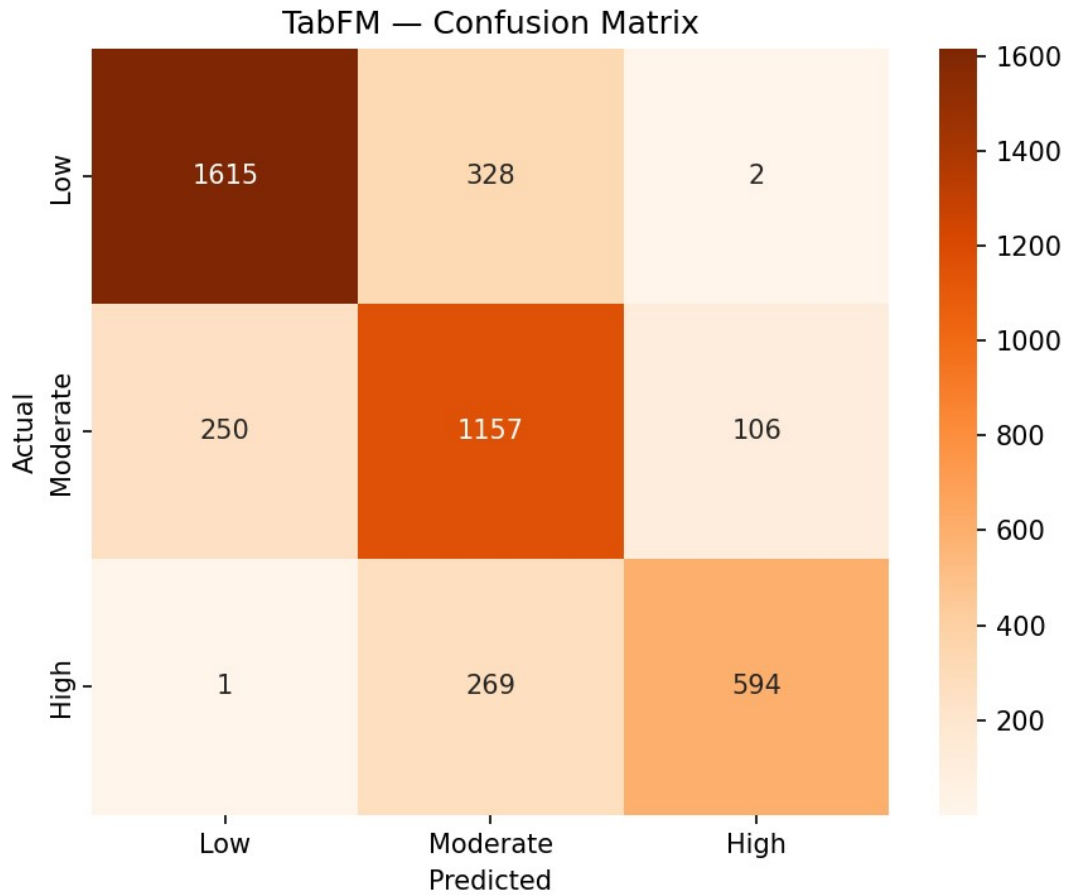


Fig. 4.3: TabFM confusion matrix

The confusion matrices (Fig. 4.1 to Fig. 4.3) show that almost all errors fell between neighbouring tiers. Only 3 records for CatBoost, 1 for TabPFN and 3 for TabFM were confused between Low and High, out of 929, 926 and 956 misclassified records respectively. The pattern of errors is described in Table 4.3.

Table 4.3: Share of records of each true tier assigned to a neighbouring tier

Model	Low predicted as Moderate	Moderate predicted as Low	Moderate predicted as High	High predicted as Moderate
CatBoost	13.9% (271)	19.0% (288)	7.3% (111)	29.6% (256)
TabPFN	14.9% (290)	17.7% (268)	9.1% (137)	26.6% (230)
TabFM	16.9% (328)	16.5% (250)	7.0% (106)	31.1% (269)

Because the tiers are ordered, the errors were also measured with ordinal-aware measures (Table 4.4).

Table 4.4: Ordinal-aware measures

Model	Cohen's kappa	Quadratic weighted kappa	Within one tier	Mean absolute error (tiers)
CatBoost	0.6586	0.8074	99.93%	0.2156
TabPFN	0.6612	0.8115	99.98%	0.2145
TabFM	0.6495	0.7987	99.93%	0.2219

Cohen's kappa was about 0.65 to 0.66 for the three models, meaning that they agreed with the true tiers much more than chance would allow. Quadratic weighted kappa was higher, about 0.80 to 0.81, because it gives less weight to errors between neighbouring tiers, and this agrees with the observation that almost all errors were of that kind. At least 99.93% of the predictions were within one tier of the true tier, and the mean absolute error was about 0.22 tiers. The ranking of the three models is the same as in Table 4.1, with TabPFN slightly ahead and TabFM slightly behind, and the differences are small.

The High tier had the lowest recall in every model (0.70, 0.73 and 0.69), and between 26.6% and 31.1% of its records were predicted as Moderate. The Moderate tier had higher recall (0.73 to 0.76) than precision (0.66 to 0.68), which means that the models assigned many records to Moderate that belonged to a neighbouring tier. The two facts are connected: records that truly belong to High but lie near the boundary are absorbed into the large Moderate group. The ordered nature of the target is a likely reason for the weaker Moderate results: the Moderate tier lies between the other two and shares a boundary with each of them, whereas Low and High each border only one tier. This study did not test that explanation with an ordinal model, so it is offered as an interpretation. High-risk records were under-rated as Moderate far more often (26.6% to 31.1%) than Moderate records were over-rated as High (7.0% to 9.1%). For a screening use, a "Moderate" prediction should therefore not be read as excluding high risk.

In practical terms, an accuracy of about 78% means that roughly 22 records in every 100 were placed in a different tier from the true one, and that almost all of these were placed in the neighbouring tier. The models were therefore useful for ranking patients by risk and for separating low from high risk, but they were not precise enough to assign the exact tier with confidence, in particular around the Moderate boundary. This is a description of performance on this dataset, and it is not a statement about clinical use.

4.1.3 Statistical significance

McNemar's test with continuity correction was applied to each pair of models on the same 4,322 records (Table 4.5).

Table 4.5: Pairwise McNemar's test

Comparison	Statistic	p-value	Significant at $\alpha = 0.05$
CatBoost vs TabPFN	0.0147	0.9037	No
CatBoost vs TabFM	1.6528	0.1986	No
TabPFN vs TabFM	2.5798	0.1082	No

None of the three differences was significant, so no difference in accuracy was detected between a grid-searched gradient-boosted model, a zero-shot model with the full training data, and a zero-shot model with a 500-row context. A non-significant result does not prove that the models are equivalent. It only shows that the data give no evidence of a difference. The comparison with TabFM was also made at unequal context sizes, which is why Section 4.2 examines the models at matched sizes.

To show how uncertain each accuracy value is, 95% Wilson score intervals were calculated from the accuracy and the number of test records (Table 4.6).

Table 4.6: Accuracy with 95% Wilson score intervals

Model	Accuracy	95% interval
CatBoost	78.51%	77.26% to 79.70%
TabPFN	78.57%	77.33% to 79.77%
TabFM	77.88%	76.62% to 79.09%

Each interval is about 2.4 percentage points wide, and the three intervals overlap almost completely, which agrees with the absence of significant differences in Table 4.5. The intervals describe only the uncertainty that comes from the finite test sample. They do not include variation from the choice of training records or from the random seed, which was not measured because each model was trained once.

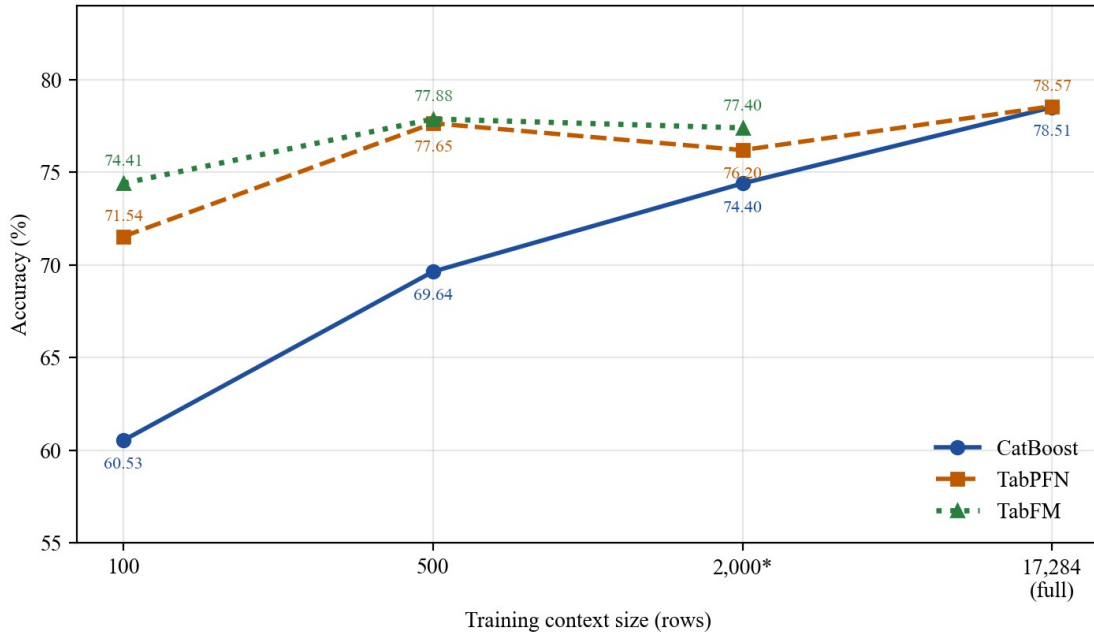
Summary for Objective 1. The three models reached close performance on the same test records. TabPFN had the highest accuracy, macro-F1 and ROC-AUC, but none of the differences was significant. All three models were weakest on the Moderate tier, and almost all errors fell between neighbouring tiers, with High-risk records most often under-rated as Moderate.

4.2 Effect of training context size

The three models were retrained with 100, 500 and 2,000 training rows (Section 3.8.5). Table 4.7 and Fig. 4.4 give the results. The 100-row and 500-row runs used the full test partition, and the 2,000-row run used a fixed subset of 500 test records for all models. The 2,000-row column can therefore be compared across models but not with the other rows in absolute terms.

Table 4.7: Accuracy and macro-F1 with reduced training context

Training rows	Test records	CatBoost acc.	TabPFN acc.	TabFM acc.	CatBoost F1	TabPFN F1	TabFM F1
100	4,322	60.53%	71.54%	74.41%	0.5745	0.7062	0.7363
500	4,322	69.64%	77.65%	77.88%	0.6768	0.7691	0.7715
2,000	500	74.40%	76.20%	77.40%	0.7260	0.7508	0.7631
Full (17,284)	4,322	78.51%	78.57%	not run	0.7763	0.7787	not run



* 2,000-row results use a 500-record test subset; the other sizes use all 4,322 test records. TabFM was not run at full context.

Fig. 4.4: Accuracy of the three models under reduced training context

At every reduced size, both foundation models scored higher than CatBoost. At 100 rows, which is under 0.6% of the training partition, CatBoost reached 60.53%, which was 11.01 points below TabPFN and 13.88 points below TabFM. The gaps were 8.01 and 8.24 points at 500 rows and 1.80 and 3.00 points at 2,000 rows, so they narrowed as the context grew, and at full context CatBoost matched TabPFN (78.51% and 78.57%). The macro-F1 values show the same picture, and the gap in macro-F1 at 100 rows (0.5745 for CatBoost against 0.7062 and 0.7363) is even larger than the gap in accuracy. This suggests that CatBoost's

errors at 100 rows were concentrated in some tiers, but per-class results were not examined at the reduced sizes.

The foundation models kept most of their full-data performance with small contexts. At 500 rows TabPFN was 0.92 points below its own full-context accuracy, whereas CatBoost was 8.87 points below its own. TabFM at 500 rows was 0.63 points below CatBoost's full-context accuracy while using 2.9% of the training records. From 100 to 500 rows CatBoost gained 9.11 points, TabPFN 6.11 and TabFM 3.47, so the tuned gradient-boosted model depended most on the amount of training data. A plausible reason is that tree splits need enough records in each region of the feature space to be reliable, whereas the foundation models bring knowledge from pretraining. This explanation was not tested in this study.

A practical way to read the same results is to ask how much data each model needed to pass an accuracy of 70%. TabPFN (71.54%) and TabFM (74.41%) passed it with only 100 training rows, whereas CatBoost was still below it with 500 rows (69.64%) and passed it only at 2,000 rows (74.40%, on the smaller test subset). The foundation models therefore needed at least five times less data to reach the same level.

TabFM scored higher than TabPFN at all three reduced sizes, but the margin shrank from 2.87 points at 100 rows to 0.23 points at 500 rows. The 1.20-point margin at 2,000 rows was measured on only 500 records, where the 95% interval for an accuracy near 76% is about ± 3.7 points, so the order of the two foundation models cannot be treated as reliable. McNemar's test was not run at the reduced sizes, so no significance claim is made for these comparisons. The full-context TabFM run did not finish (Section 4.4), which is why that cell is empty.

Summary for Objective 2. The amount of training data mattered most for CatBoost. With 100 to 500 rows both foundation models were clearly better, the gap narrowed as the context grew, and with the full training partition CatBoost matched TabPFN. TabFM's rank against TabPFN was not reliable at any size.

4.3 Explainability of the predictions

4.3.1 Relationship between the features and the risk tier in the data

Before the SHAP results are presented, the relationships in the data itself are described (Section 3.8.2). Table 4.8 gives the mean of each continuous predictor in each risk tier, and Table 4.9 gives the proportion of records with each binary flag.

Table 4.8: Mean of the continuous predictors by risk tier

Predictor	Low	Moderate	High	Spearman correlation with risk tier
stress_level	15.40	19.05	22.18	+0.329
iron (µg/dL)	108.34	94.82	82.64	-0.314
total_protein (g/dL)	7.39	7.13	6.91	-0.304
vitamin_d (ng/mL)	31.23	26.92	23.18	-0.262
alt_liver (U/L)	22.20	26.64	31.73	+0.254
calcium (mg/dL)	9.44	9.24	9.08	-0.226
manganese (µg/L)	8.56	7.85	7.09	-0.184
body_water_content (%)	56.05	54.62	53.22	-0.135
age (years)	31.73	33.06	34.90	+0.116
hair_texture	50.02	50.46	49.97	+0.003
total_keratine	49.91	50.10	49.40	-0.006

Table 4.9: Proportion of records with each binary flag, by risk tier

Flag	Low	Moderate	High
stress	34.2%	42.8%	49.9%
chemical_use	33.9%	42.6%	48.6%
family_hair_fall_history	28.9%	37.1%	42.5%
late_night_sleep	36.5%	41.3%	46.4%
sleep_disturbance	31.6%	36.2%	41.5%
chronic_illness	25.1%	31.7%	37.5%
water_reason	27.0%	30.6%	34.6%
anemia	19.8%	26.3%	32.5%

Seven biomarkers (stress_level, iron, total_protein, vitamin_d, alt_liver, calcium and manganese) had the strongest rank correlations with the risk tier, between 0.18 and 0.33 in absolute value, and the direction was as expected for the constructed relationships: higher stress_level and alt_liver with higher risk, and higher iron, protein, vitamin D, calcium and manganese with lower risk. Every binary flag was more common in the High tier than in the Low tier. The two composite hair-condition scores, total_keratine and hair_texture, had correlations close to zero and almost identical means in the three tiers. All correlations are moderate at most, and no single feature separates the tiers, which agrees with the statement in Section 1.1 that risk arises from a combination of factors.

4.3.2 Global feature importance

Fig. 4.5 to Fig. 4.7 rank the features by mean absolute SHAP value. The CatBoost plot covers the full test partition and adds the contributions of the three classes. The TabPFN and TabFM plots cover only the sampled records (30 and 20) and show the High risk class.

The bar lengths are on different scales, so the models are compared by rank and not by magnitude. Table 4.10 lists the ten most important features in each model.

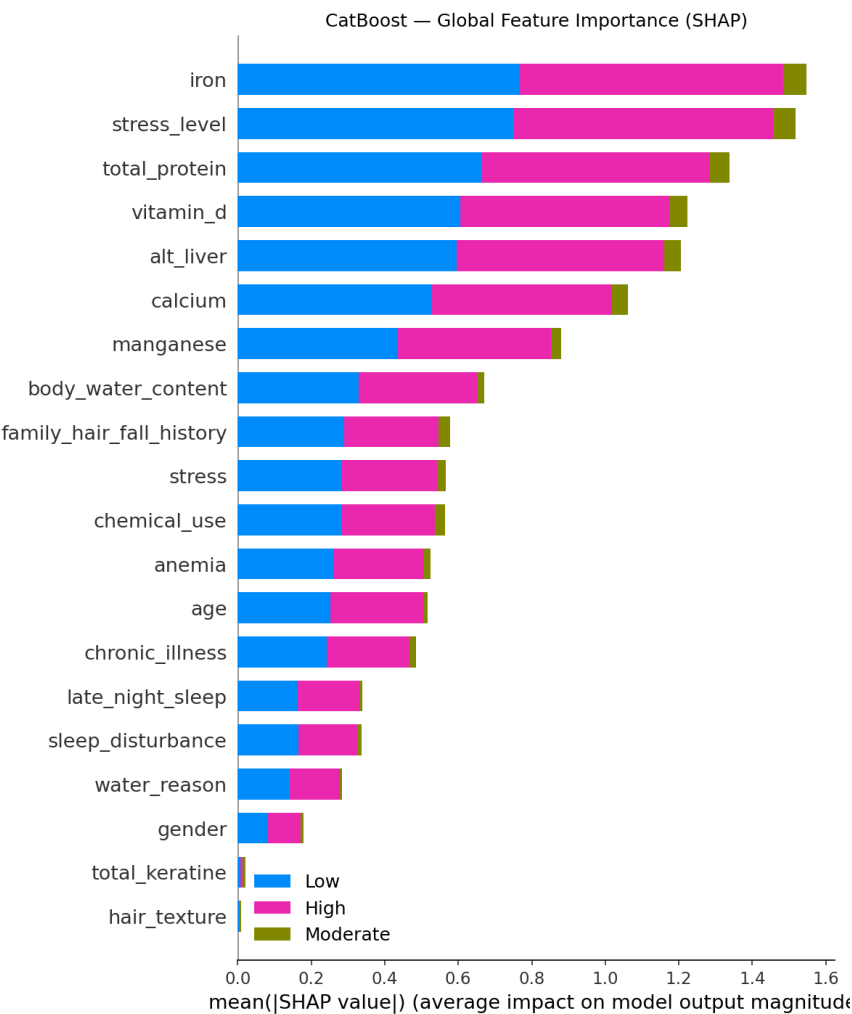


Fig. 4.5: CatBoost global feature importance (SHAP)

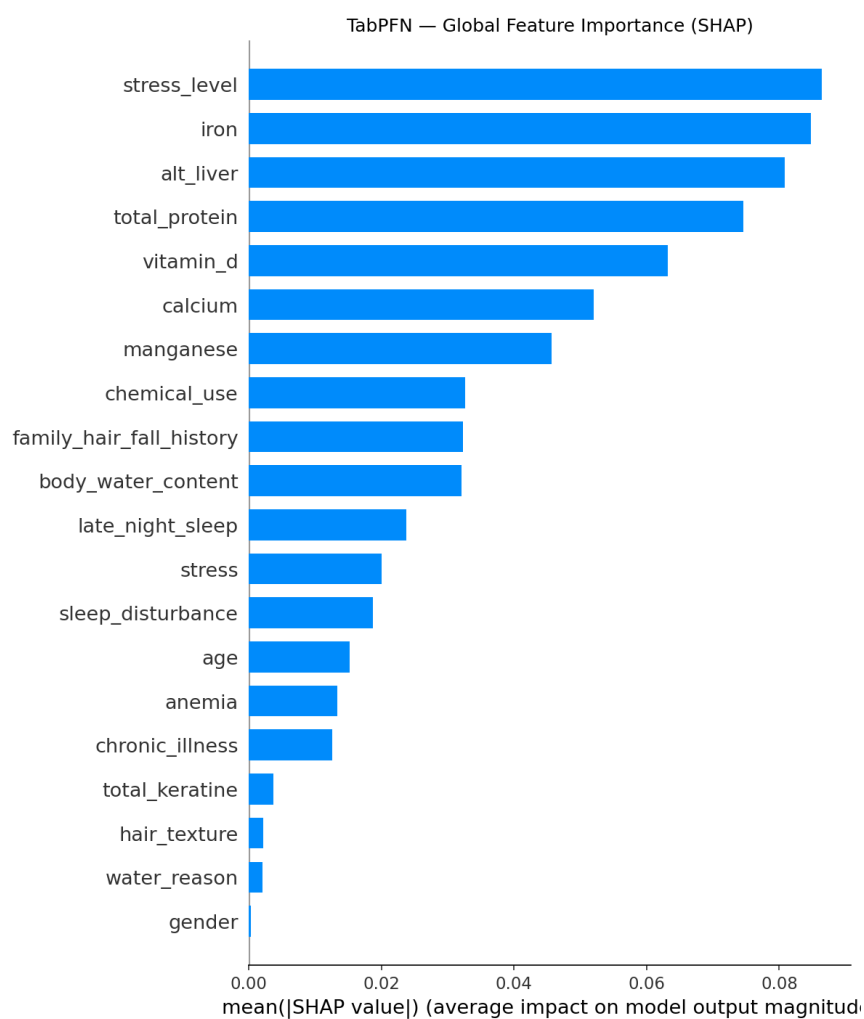


Fig. 4.6: TabPFN global feature importance (SHAP)

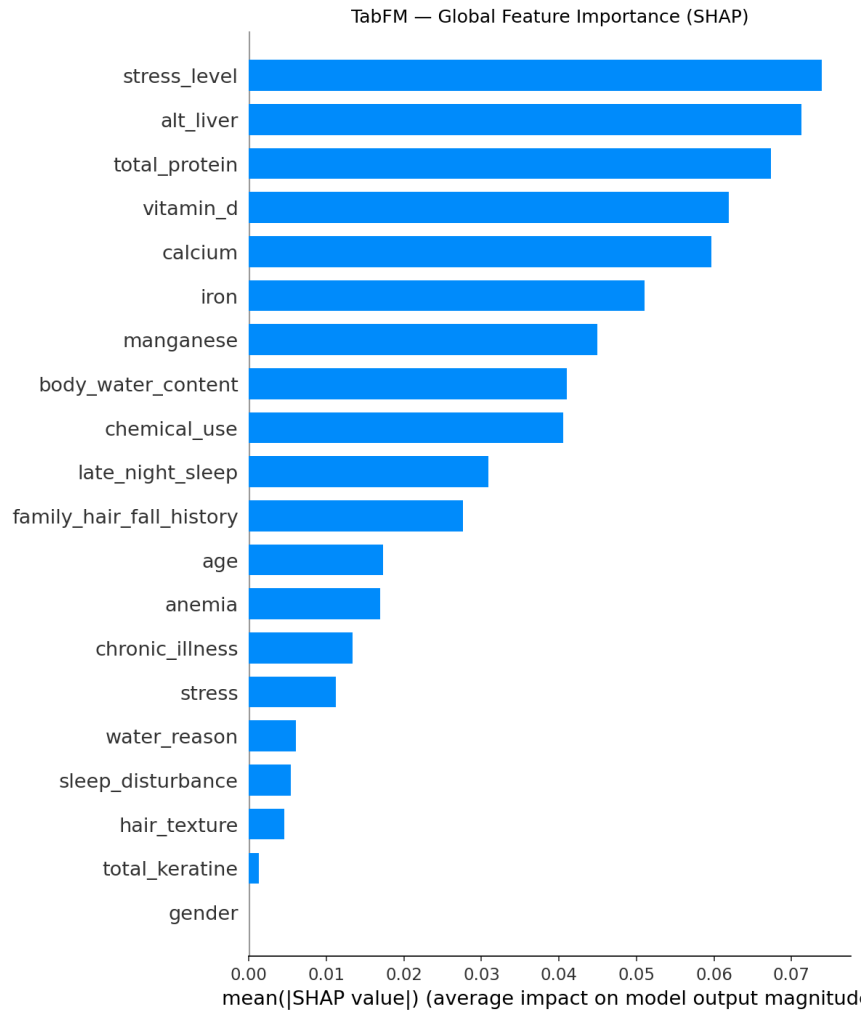


Fig. 4.7: TabFM global feature importance (SHAP)

Table 4.10: Ten most important features in each model

Rank	CatBoost	TabPFN	TabFM
1	iron	stress_level	stress_level
2	stress_level	iron	alt_liver
3	total_protein	alt_liver	total_protein
4	vitamin_d	total_protein	vitamin_d
5	alt_liver	vitamin_d	calcium
6	calcium	calcium	iron
7	manganese	manganese	manganese
8	body_water_content	chemical_use	body_water_content
9	family_hair_fall_history	family_hair_fall_history	chemical_use
10	stress	body_water_content	late_night_sleep

The same seven continuous biomarkers (iron, stress_level, total_protein, vitamin_d, alt_liver, calcium and manganese) were the seven most important features in all three models, and manganese was seventh in each. The models differed mainly in the order within this group. For example, iron was first for CatBoost, second for TabPFN and sixth for TabFM. Because the foundation-model values came from only 20 to 30 records,

differences in this order should not be interpreted. Only the shared set of leading features is reported as a finding. The composite scores `total_keratine` and `hair_texture` contributed almost nothing in all three models, and `gender` also contributed very little.

This set of seven features is exactly the set with the strongest rank correlations in Table 4.8, and the two features with almost no correlation are also the two with almost no SHAP importance. The models therefore relied on the features that are most strongly associated with risk in the data, and this gives a consistency check that does not depend on the models themselves.

4.3.3 Direction of the effects

The beeswarm plots (Fig. 4.8 to Fig. 4.10) show how feature values moved the predicted probability of the High risk class.

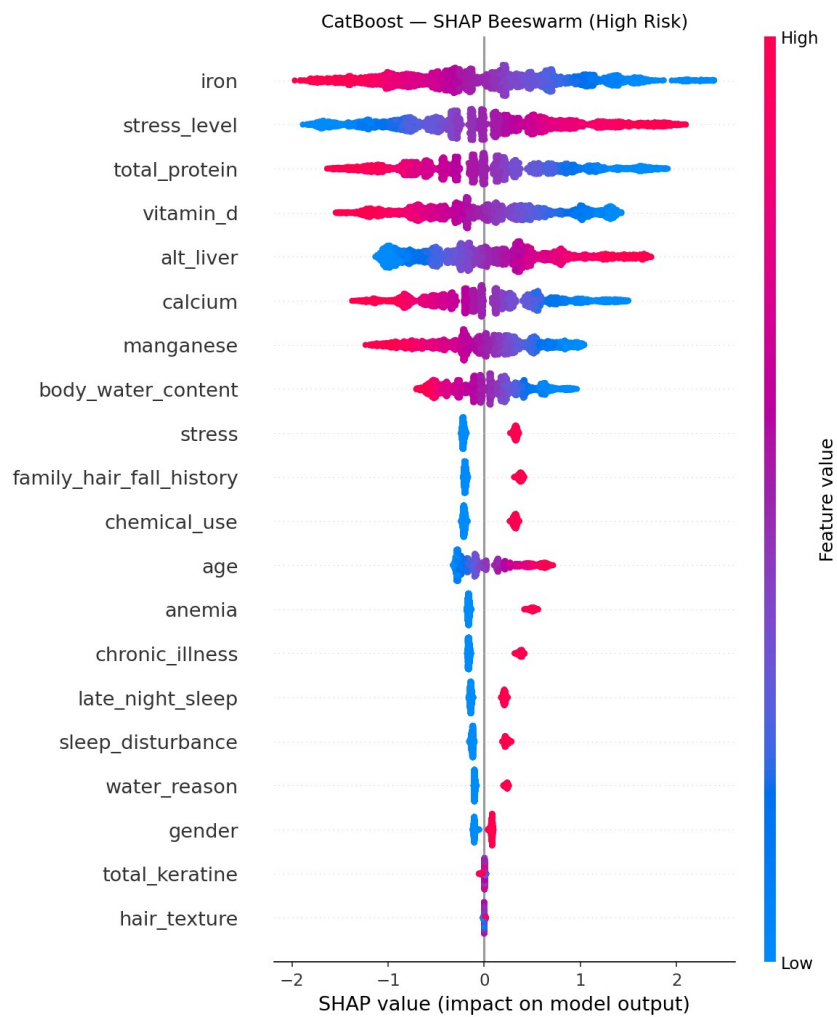


Fig. 4.8: CatBoost SHAP beeswarm plot (High risk)

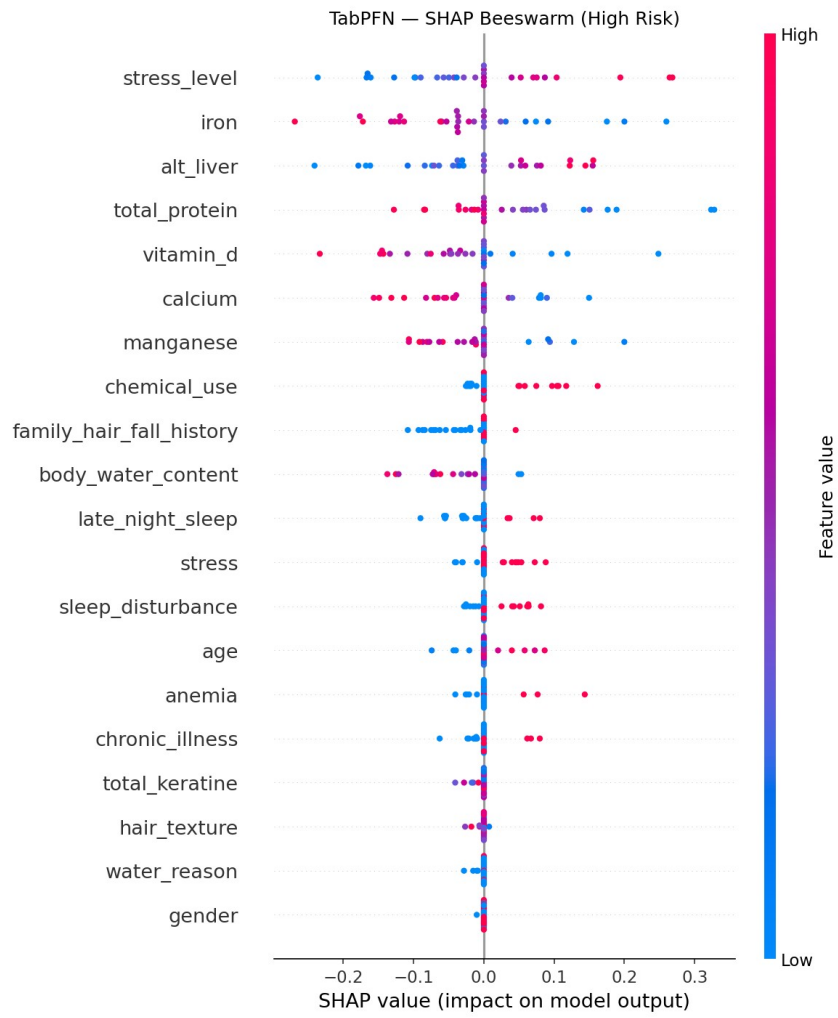


Fig. 4.9: TabPFN SHAP beeswarm plot (High risk)

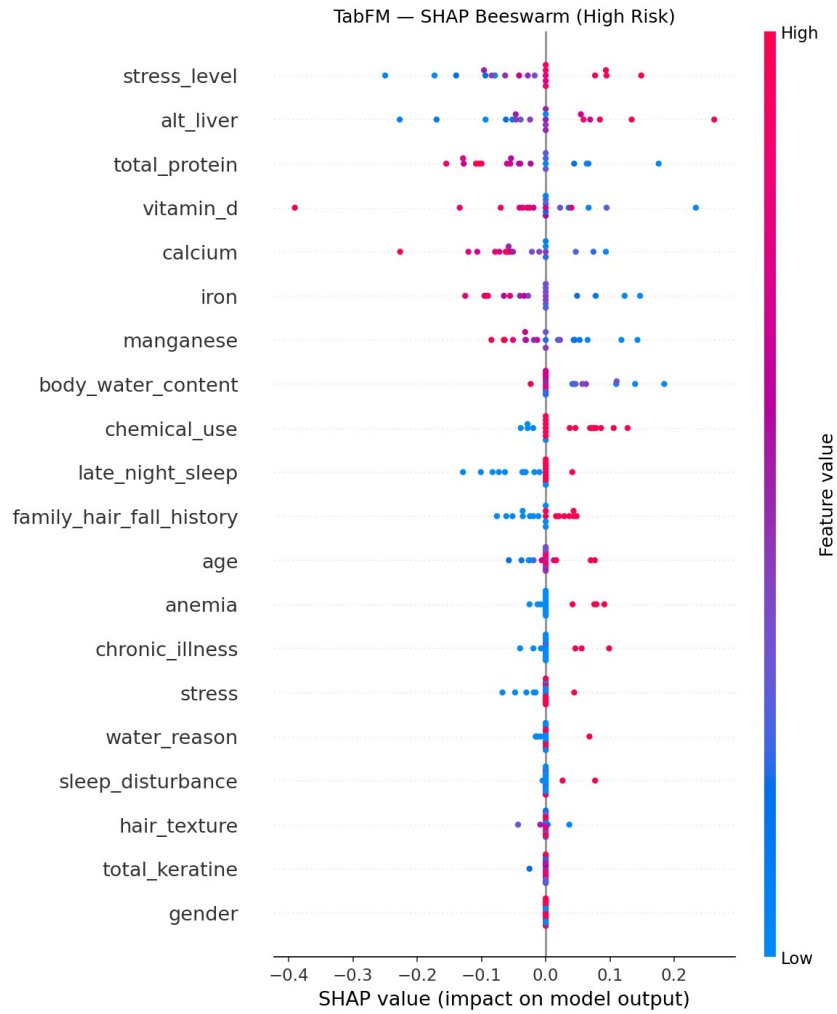


Fig. 4.10: TabFM SHAP beeswarm plot (High risk)

The colour of each point in a beeswarm plot shows the value of the feature for that record (red for high values and blue for low values), and its position on the horizontal axis shows how much that value raised (right) or lowered (left) the predicted probability of High risk. A feature whose red points lie on the left and blue points on the right, such as iron, therefore lowers the predicted risk when its value is high. In all three models, high values of stress_level and alt_liver pushed the prediction toward High risk, and high values of iron, total_protein, vitamin_d, calcium and manganese pushed it away. For CatBoost, higher body_water_content also lowered the High risk contribution, and higher age and every binary risk flag (stress, family_hair_fall_history, chemical_use, anemia, chronic_illness, late_night_sleep, sleep_disturbance and water_reason) raised it. The TabPFN and TabFM plots showed the same direction for the leading biomarkers. These directions agree with the signs of the correlations in Table 4.8 and the differences in Table 4.9.

4.3.4 Differences between the rankings

The rankings in Table 4.10 differ outside the shared set of seven features. TabPFN and TabFM placed `chemical_use` eighth and ninth, whereas CatBoost placed it lower, and CatBoost placed `family_hair_fall_history` and the stress flag ninth and tenth. Within the leading seven, `iron` was first for CatBoost but sixth for TabFM, and `stress_level` was first for both foundation models. These differences may reflect the different ways in which the models use the features, but they may equally reflect the small samples (30 and 20 records) on which the foundation-model values are based, and Kernel SHAP estimates for features of similar size can change order between runs. The study therefore does not claim that any of these rank differences is a property of a model. The finding that can be relied on is the shared set of leading features and the shared direction of their effects.

4.3.5 Patient-level explanations

Waterfall plots show how the features moved one patient's predicted High risk score from the baseline value to the final value (Fig. 4.11 to Fig. 4.13). The three plots show different patients, because the CatBoost plot uses the first test record and the two foundation-model plots each use the first record of a different random sample, so they should not be compared with each other. They illustrate what a clinician would see for one patient: a short list of features that raised the score and features that lowered it, each with its size.

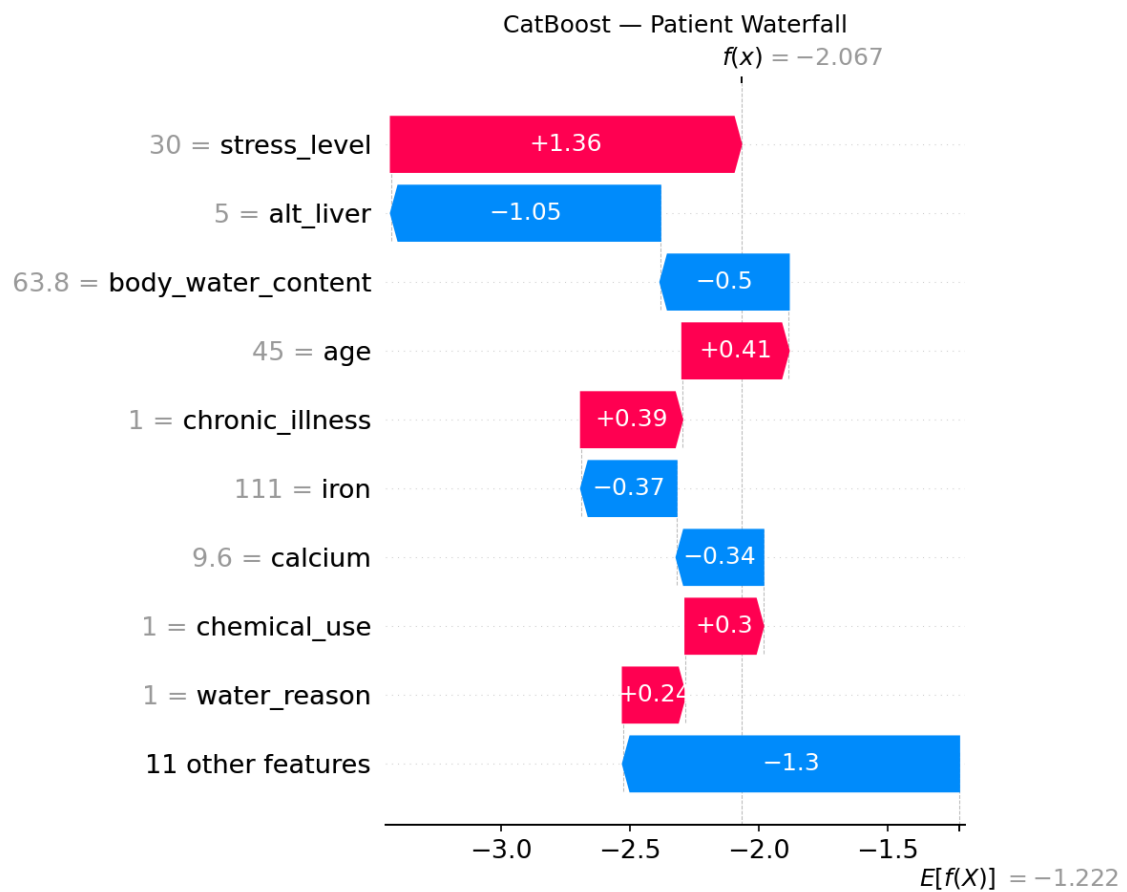


Fig. 4.11: CatBoost patient-level explanation

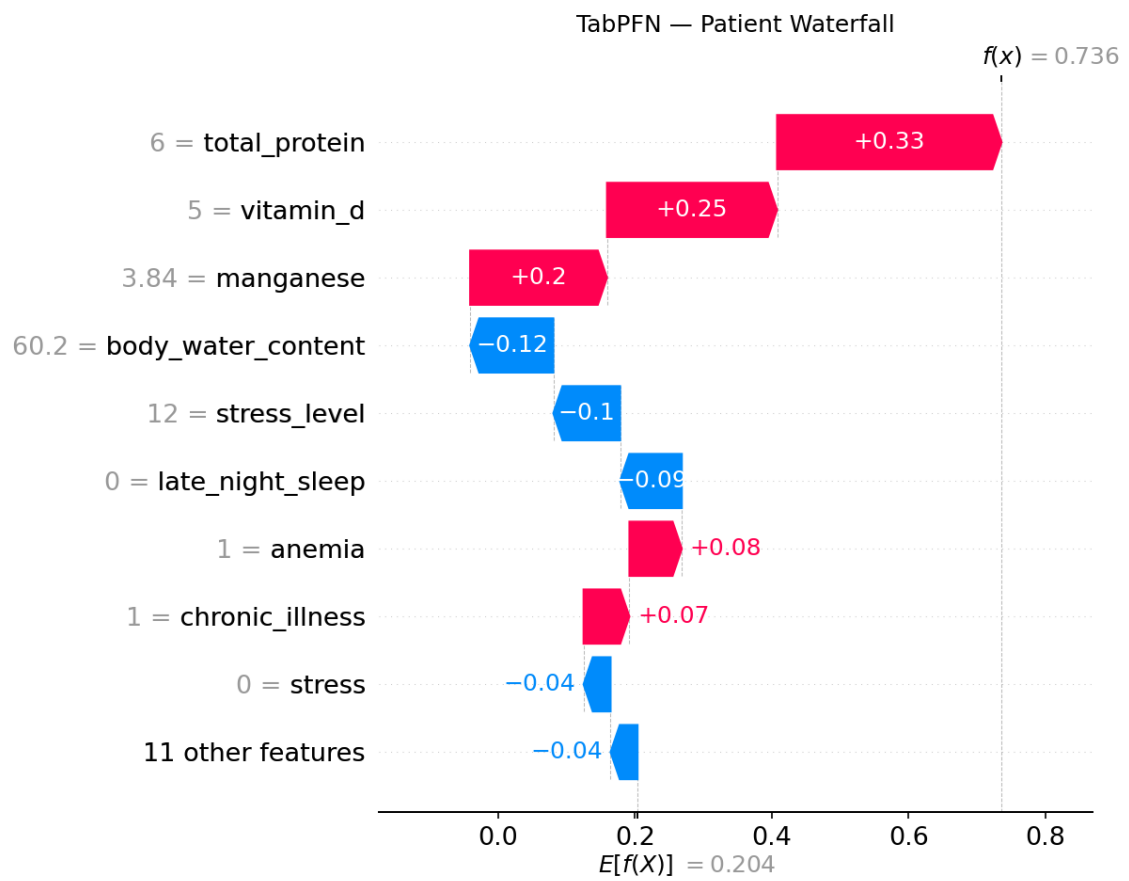


Fig. 4.12: TabPFN patient-level explanation

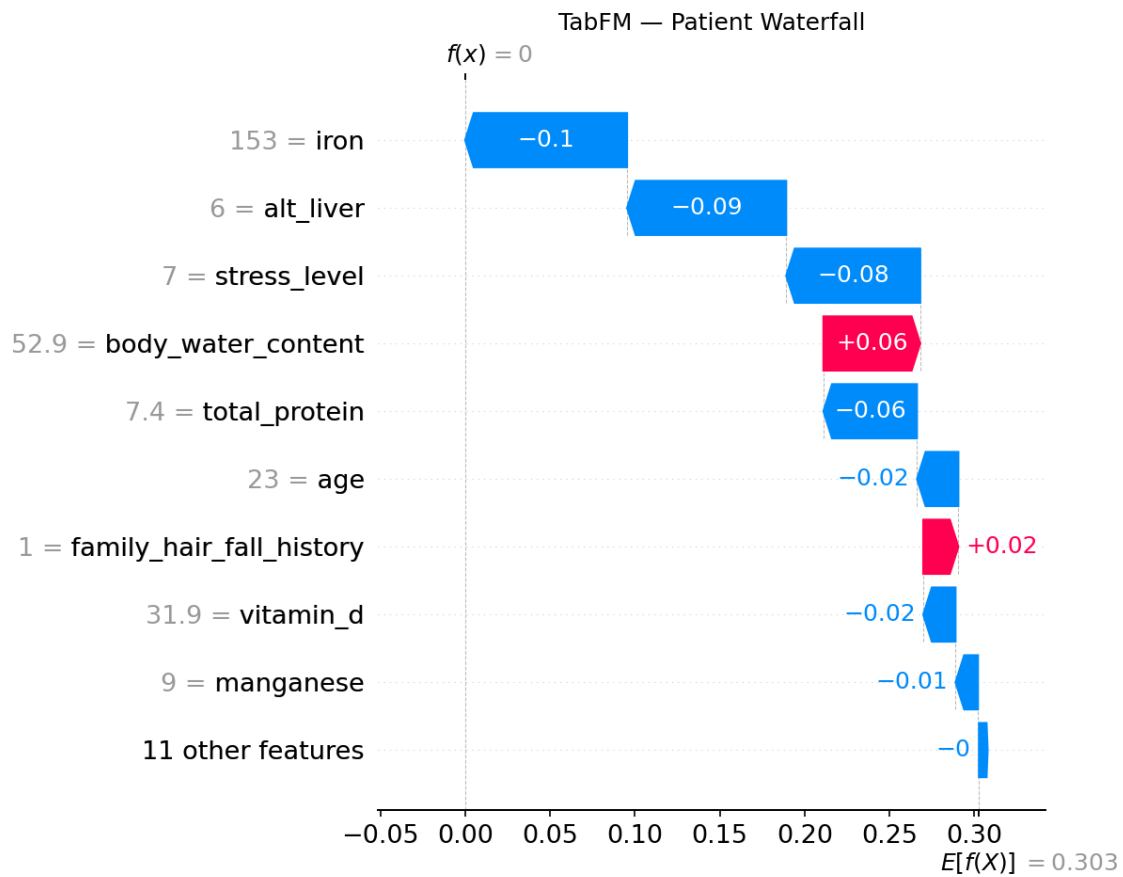


Fig. 4.13: TabFM patient-level explanation

Summary for Objective 3. The same seven biomarkers led the SHAP rankings of all three models, with the same direction of effect. They are also the seven features most strongly correlated with the risk tier in the data, and the two hair-condition scores that carry no association with the tier had almost no importance. The explanations therefore agree with the data. The foundation-model explanations rest on 20 to 30 records, so only the shared set of leading features was interpreted.

4.4 Computational cost and implementation limitations

4.4.1 TabFM runtime

TabFM (version 1.0.0, PyTorch backend) was much slower at prediction than CatBoost and TabPFN, and its running time rose steeply with the size of the training context (Table 4.11). With a 500-row context, predicting the 4,322 test records took approximately 8 minutes. With a 2,000-row context, predicting only 500 test records took approximately 15 minutes. That is about 1.8 seconds per record compared with about 0.11 seconds per record at 500 rows, so a context four times larger made each prediction roughly sixteen times slower. An attempt to use the full 17,284-row context, run in batches of 50 records with

results saved after each batch, did not finish within the session limits of the cloud platform. TabFM was therefore evaluated with a 500-row context, the largest tested size that allowed a complete evaluation of the full test partition. A single ensemble member ($n_estimators = 1$) was used throughout, because the default ensemble setting multiplied the cost of every prediction.

Table 4.11: Approximate TabFM prediction time by context size

Training context	Test records	Approximate time	Approximate time per record
500 rows	4,322	8 minutes	0.11 s
2,000 rows	500	15 minutes	1.8 s
17,284 rows	4,322	Did not finish	–

The practical meaning is that a model that needs no tuning is not necessarily cheap to use. CatBoost had a training cost and a very small prediction cost, and the foundation models had almost no training cost but a large prediction cost that depends on the context. The choice between them therefore depends on how many predictions are needed and how large the context is.

4.4.2 Software problems and workarounds

Three problems arose while running the models.

1. `PermutationExplainer` failed. This SHAP method, which had been planned for TabPFN and TabFM, failed with a type-resolution error in `numba`, the library that shap uses to speed up its masking step. The failure was reproduced on both Google Colaboratory and Kaggle Notebooks. It continued after `numba`'s JIT compilation was disabled with the `NUMBA_DISABLE_JIT` setting and after `numba` and `llvmlite` were reinstalled. `KernelExplainer` was used instead. It estimates the same Shapley values by querying the model directly, but it is slower, which is why the foundation-model explanations were limited to 30 and 20 test records and a reduced training context. The consequence was a change of method for two of the three models, which is why the SHAP results of CatBoost (exact, all test records) and of the foundation models (estimated, 20 to 30 records) are not treated as equally precise.
2. TabFM failed on single-row input. The `predict_proba` method failed when called with one row, which `KernelExplainer` does routinely. A single-row query was therefore duplicated into a two-row batch, and only the first output row was kept.

This does not change the prediction for the original record. It may be an upstream defect, but this was not checked against the reported defects in [10].

3. TabPFN checkpoint loading failed. Recent PyTorch versions load checkpoints with `weights_only` set to `True` by default, which rejected the TabPFN weights. `torch.load` was wrapped so that `weights_only` was `False` before the model was created. TabPFN's licence token was read from the Kaggle secrets store and not written into the notebook.

These problems came from the recency of the software and not from the modelling approaches. TabFM's runtime growth and its single-row failure were not visible from its documentation and were found by testing, which required changing one variable at a time (context size, batch size and backend) to separate design limits from defects.

Summary for Objective 4. CatBoost and TabPFN ran without difficulty. TabFM was slow, its prediction time grew steeply with the training context, and its full-context evaluation could not be completed. The planned SHAP method failed and was replaced, TabFM failed on single-row input, and TabPFN needed a checkpoint-loading workaround.

4.5 Discussion

4.5.1 Plausibility of the findings

The three models reached close accuracy (77.88% to 78.57%) on a task with three imbalanced tiers, and most errors fell between neighbouring tiers, which is what would be expected when risk changes gradually with the input values. The main features found by SHAP (iron, stress level, total protein, vitamin D, liver enzyme ALT, calcium and manganese) and the direction of their effects agree with the relationships used to construct the dataset (Section 3.7) and with the correlations measured in the data (Section 4.3.1). The result should therefore be read as showing that all three models recovered the relationships built into the data. It is not new clinical evidence, because the data were not collected from a clinical registry. For the same reason, the accuracy values show how well the models perform on this dataset and cannot be taken as the accuracy of a diagnostic tool.

The very small contribution of `total_keratine` and `hair_texture` is also plausible in this setting. They are composite scores of hair condition and not laboratory measurements, and the data show almost no association between them and the risk tier (Table 4.8).

A ceiling of about 78% accuracy for all three models, whatever their design, suggests that the limit is set by the information in the features and their overlap between tiers, and not by the choice of model. Three very different methods (tuned trees and two transformers) reached the same level, which is what would be expected if all of them had extracted most of the available signal. This interpretation was not tested directly, for example with additional model families.

4.5.2 Comparison with similar studies

A direct comparison of accuracy values with earlier hair loss studies is not possible, because the studies used different data, targets and metrics. The comparison below is therefore about findings and not about numbers.

- Sai et al. [3] found that an ensemble outperformed individual classifiers, and Patel et al. [4] found that Random Forest outperformed XGBoost, CatBoost and LightGBM. Both concern classical models on different datasets. The present study did not include these methods, but it adds that a tuned CatBoost and two zero-shot foundation models were not significantly different in accuracy on this dataset (Table 4.5). This agrees with the view in Section 2.8 that the best model depends on the dataset.
- TabFM has been reported to outperform heavily tuned gradient-boosted models on the TabArena benchmark in a zero-shot setting [9]. In the present study TabFM was not significantly different from CatBoost (with a much smaller context), and with the same small contexts both foundation models scored higher than CatBoost (Table 4.7). This is consistent with a data-efficiency advantage, but at the full training size no advantage over CatBoost was detected, so the findings support the benchmark report only partly. The independent evaluation [10] found TabFM competitive with XGBoost, Random Forest and TabPFN, and the present result for TabFM against TabPFN and CatBoost points in the same direction.
- TabPFN was reported to outperform tuned tree ensembles on datasets of up to 10,000 samples [20]. The present training partition (17,284 rows) is larger than that range, and TabPFN matched CatBoost there, without exceeding it. This suggests that the advantage of TabPFN is greatest with small data and disappears as the training set grows, which agrees with the scaling results in Section 4.2 and with its design for small problems [8].

- The studies of Schwartz-Ziv and Armon [17] and Grinsztajn et al. [18] found that tuned trees remain very strong on tabular data. The present result, in which CatBoost was not beaten at full data size, is consistent with them. The result at small data sizes, in which the foundation models were clearly better, shows the situation in which a pretrained model can add value.
- Chen et al. [5] observed that neural models gained accuracy but lost interpretability. Here SHAP explanations could be produced for CatBoost and for both transformer-based models, although with more limited samples for the latter, so interpretability did not have to be given up.
- Widowati et al. [1] stressed that transformations must be kept inside cross-validation folds. In the present study the only preprocessing was the removal of identifiers and a fixed coding of gender, so no fitted transformation could leak information, and CatBoost's tuning was done by cross-validation within the training partition.

4.5.3 Implications and cautions

The results suggest that, on this dataset, zero-shot foundation models can be used without a tuning stage and still reach a performance comparable to a tuned CatBoost, and that they are more useful when little training data is available. Three practical consequences follow. First, when a hair fall risk model has to be built from a few hundred labelled records, a pretrained model is a sensible starting point, because the accuracy of CatBoost fell by about nine points from the full data to 500 rows whereas that of TabPFN fell by less than one point. Second, when a large labelled dataset and a fast prediction service are available, a tuned gradient-boosted model performs as well as the foundation models and is much cheaper to run, so there is no accuracy reason to prefer the larger models. Third, whichever model is used, the explanation should be produced and checked against the data, as was done in Section 4.3.1, because a model can be accurate for reasons that a clinician would not accept. Against this, TabFM was slow and could not be run with the full context, and the explanations for the foundation models rest on small samples. The main limitations (a constructed dataset, one data split, the unequal contexts and test sets, the small SHAP samples, and the use of the test partition for CatBoost's early stopping) are discussed in Chapter 5.

4.5.4 Relation of the results to the research questions

Research Question 1 asked whether the zero-shot foundation models can match a tuned CatBoost and whether the differences are significant. The answer from Section 4.1 is that no significant difference in accuracy was detected between any pair of models. Research Question 2 asked how the training context size affects the models. Section 4.2 showed that CatBoost lost much more accuracy than the foundation models when the context was reduced, and that the gap closed as the context grew. Research Question 3 asked which features drive the predictions and whether they agree across models and are plausible. Section 4.3 showed that the same seven biomarkers led all three rankings, with the same direction of effect, in agreement with the relationships measured in the data. Research Question 4 asked about computational cost and limitations. Section 4.4 showed that TabFM's prediction time grew steeply with context size and that several software problems needed workarounds. Chapter 5 draws the conclusions from these answers.

CHAPTER 5

CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

This study set out to benchmark CatBoost, TabPFN and TabFM for explainable, multi-tier (Low, Moderate, High) hair fall risk stratification from structured clinical and lifestyle data. It was motivated by three gaps identified in Section 1.2: the evidence on the best model for hair fall data was inconsistent, tabular foundation models had not been tested on this problem, and complex models were hard to interpret. The four research questions of Section 1.3 are answered below, using the results of Chapter 4.

Predictive performance (Research Question 1). The three models reached close results on the 4,322-record test partition. Accuracy was 78.51% for CatBoost, 78.57% for TabPFN and 77.88% for TabFM, and macro-F1 was 0.7763, 0.7787 and 0.7715. McNemar's test found no significant difference between any pair of models at the 0.05 level. On this dataset, therefore, no advantage of the tuned gradient-boosted model over the two zero-shot models was detected, even though only CatBoost was tuned and TabFM used a 500-row context, under 3% of the training records. This does not prove that the models are equivalent.

Effect of training context size (Research Question 2). When the training context was reduced to 100 and 500 rows, both foundation models scored clearly higher than CatBoost. At 100 rows the accuracy was 60.53% for CatBoost, 71.54% for TabPFN and 74.41% for TabFM, and at 500 rows it was 69.64%, 77.65% and 77.88%. The gap narrowed as the context grew, and CatBoost matched TabPFN with the full training partition. At 500 rows, CatBoost was 8.87 points below its full-data accuracy and TabPFN was 0.92 points below its own. The tuned gradient-boosted model therefore depended most on the amount of training data. Because two independently developed foundation models behaved in the same way on this dataset, the behaviour does not seem to be specific to one design, although only one dataset was tested. The ranking of TabFM and TabPFN with each other was not reliable at any size.

Explainability (Research Question 3). SHAP explanations were produced for all three models. The same seven features (iron, stress_level, total_protein, vitamin_d, alt_liver, calcium and manganese) were the most influential in each, and the direction of their effects was the same: higher stress_level and alt_liver values raised the predicted High risk

contribution, and higher iron, total_protein, vitamin_d, calcium and manganese values lowered it. This agrees with the relationships used to build the dataset, so it shows that the models recovered those relationships and it is not new clinical evidence.

Computational cost and implementation (Research Question 4). CatBoost and TabPFN ran without difficulty, but TabFM was slow and its running time rose steeply with context size. Its full-context evaluation could not be completed, so a 500-row context was used. The runs also exposed software problems: the planned PermutationExplainer failed because of a numba error and was replaced with KernelExplainer, TabFM failed on single-row input, and TabPFN's checkpoint loading needed a workaround. These problems came from the recency of the software and not from the modelling approaches.

In summary, on this dataset a tuned gradient-boosted model and two zero-shot foundation models gave comparable accuracy, the foundation models were much less dependent on the amount of training data, and all three could be explained with SHAP. This meets the general objective of the study, within the limits described in Section 5.2.

The problem stated in Section 1.2 was that the evidence on the best model for hair fall data was inconsistent, that tabular foundation models had not been tested on it, and that complex models were hard to interpret. The general objective of Section 1.4 was to benchmark the three models for explainable multi-tier hair fall risk stratification, and each of the four specific objectives was met by the analyses in Sections 4.1 to 4.4. The comparison was made on one dataset, so the conclusions describe the behaviour of the models on this dataset and should not be generalised beyond it.

The contribution of the study can be stated in four points. It is a benchmark of two independently developed tabular foundation models against a tuned gradient-boosted model on multi-tier hair fall risk classification, a comparison that the reviewed literature did not contain (Section 2.8). It shows, with matched training contexts, how the models differ when data are limited. It applies one explanation method to three different architectures and checks the explanations against the relationships measured in the data (Section 4.3). It also documents the runtime and software problems met with a recently released model, which may help others who plan to use it.

5.2 Limitations of the study

The conclusions must be read with the following limitations.

1. The dataset was assembled from two public sources and adjusted with domain-informed relationships (Section 3.7). It was not drawn from a clinical registry, so the results show how the models perform on this dataset and are not the validation of a diagnostic tool.
2. TabFM used a 500-row context while CatBoost and TabPFN used 17,284 rows, so no evaluation compares all three at full scale. The 2,000-row run used a 500-record test subset, so it is less precise and not directly comparable with the other sizes. No significance tests were run at reduced contexts.
3. The SHAP analyses of TabPFN and TabFM used a reduced training context and only 30 and 20 test records, so their attributions are less certain than CatBoost's exact values on the full test partition.
4. CatBoost's final fit used the test partition to monitor early stopping, so the test data influenced the number of trees selected (Section 3.8.4). A separate validation split would be cleaner.
5. All results come from one train/test split with one random seed. Confidence intervals were given only for accuracy (Table 4.6) and describe the uncertainty from the finite test sample, not the variation between splits or seeds.
6. The Moderate tier was the hardest to classify in all three models. The study offered an explanation based on the ordered structure of the target but did not test an ordinal model.

5.3 Recommendations

Recommendations for practice. These follow from the findings and apply to structured data of a similar kind.

1. Where labelled data are limited, a zero-shot tabular foundation model such as TabPFN is a reasonable first choice, since it reached the accuracy of the tuned CatBoost without tuning and lost much less accuracy when the training data were reduced.
2. The running time of TabFM should be measured at the intended context size before an evaluation is planned, because its cost grew steeply with the context.

3. Feature-level explanations such as SHAP should accompany any risk prediction, and in this study they were available for all three models. Any use in a health setting should be as decision support and only after validation on real clinical data.

Recommendations for further research. These address the limitations above.

1. Evaluate TabFM with the full training context when its software or the available hardware allows, so that all three models can be compared under identical conditions.
2. Repeat the context-size experiment on the full test partition at every size, with several random seeds and McNemar's test at each size, to find out whether the differences between the foundation models are reliable.
3. Use repeated cross-validation or several random splits with confidence intervals, and a separate validation partition for CatBoost's early stopping.
4. Compute SHAP values for the foundation models on more test records and with larger contexts, so that the feature order can be compared more closely.
5. Compare ordinal-aware models with the multi-class approach, to see whether they improve the classification of the Moderate tier.
6. Validate the framework on real clinical hair loss data when ethically cleared data become available.
7. Study ensembles of the three models, and distillation of the foundation models into lightweight models, since TabPFN and TabFM need their training data at prediction time.

Each recommendation follows from a finding. The first recommendation for practice rests on the results of Section 4.1 and Section 4.2, the second on the runtimes in Section 4.4, and the third on the explanations in Section 4.3. Each recommendation for further research addresses one of the limitations in Section 5.2: the unequal contexts, the small significance tests at reduced sizes, the single split, the small SHAP samples, the Moderate tier, the constructed dataset and the untested ensembles. The recommendations are limited to the scope of this study and to structured data of a similar kind.

REFERENCES

- [1] A. Widowati, S. A. Yuniarto, and M. A. R. Syaipullah, "Explainable machine learning decision support system for multi-class hair loss severity classification," *International Journal of Technology in Education and Science*, vol. 10, no. 1, pp. 115–136, 2026.
- [2] M. Sayyad and S. Midhunchakkaravarthy, "Machine learning-based diagnostic model for Alopecia Areata prediction," *Journal of Dermatological Informatics*, vol. 15, no. 2, pp. 45–52, 2023.
- [3] C. N. V. Sai, E. Archana, B. Vivek, B. Dhanwanth, and V. K. S., "Enhancing Hairfall Prediction: A Comparative Analysis of Individual Algorithms and An Ensemble Method," *International Journal on Recent and Innovation Trends in Computing and Communication*, vol. 11, no. 6s, pp. 499–508, 2023.
- [4] S. Patel, V. Gupta, and N. Mehta, "Predictive Modeling of Hair Fall using Decision Tree and Random Forest Architectures," *EAI Endorsed Transactions on Pervasive Health and Technology*, vol. 11, pp. e1–e10, 2025.
- [5] K. Chen, L. Zhang, and R. Wang, "Predicting Hair Loss with AI: A Deep Learning Framework Combining Genetic and Scalp Health Data," in *Integrating AI, Machine Learning, and IoT in Bioinformatics: Innovations in Biotech and Medical Research*, CRC Press, pp. 100–107, 2024.
- [6] C. S. Shakeel, S. J. Khan, B. Chaudhry, S. F. Aijaz, and U. Hassan, "Classification Framework for Healthy Hairs and Alopecia Areata: A Machine Learning (ML) Approach," *Computational and Mathematical Methods in Medicine*, vol. 2021, Article ID 1102083, 2021.
- [7] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018, pp. 6638–6648.
- [8] N. Hollmann, S. Müller, K. Eggenberger, and F. Hutter, "TabPFN: A transformer that solves small tabular classification problems in a second," in *Proc. 11th Int. Conf. Learning Representations (ICLR)*, 2023.
- [9] W. Kong and A. Das, "Introducing TabFM: A zero-shot foundation model for tabular data," *Google Research Blog*, Jun. 2026. [Online]. Available: <https://research.google/blog/introducing-tabfm-a-zero-shot-foundation-model-for-tabular-data/>. [Accessed: DD Mon. YYYY].
- [10] devYRPauli, "tabfm-evaluation: Independent 3-machine evaluation of Google TabFM," *GitHub repository*, Jul. 2026. [Online]. Available: <https://github.com/devYRPauli/tabfm-evaluation>. [Accessed: DD Mon. YYYY].
- [11] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 4765–4774.

- [12] J. Anderson and M. Roberts, "Clinical feature analysis and survey-driven lifestyle metrics for predicting hair loss severity," *Applied Medical Informatics Review*, vol. 18, no. 1, pp. 78–89, 2025.
- [13] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [14] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [15] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [16] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 3146–3154.
- [17] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022.
- [18] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 507–520.
- [19] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.
- [20] N. Hollmann et al., "Accurate predictions on small data with a tabular foundation model," *Nature*, vol. 637, pp. 319–326, 2025.
- [21] L. S. Shapley, "A value for n-person games," in *Contributions to the Theory of Games*, vol. II, H. W. Kuhn and A. W. Tucker, Eds. Princeton, NJ, USA: Princeton Univ. Press, 1953, pp. 307–317.
- [22] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.
- [23] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.
- [24] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [25] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.
- [26] E. Frank and M. Hall, "A simple approach to ordinal classification," in *Machine Learning: ECML 2001*, ser. Lecture Notes in Computer Science, vol. 2167. Berlin, Germany: Springer, 2001, pp. 145–156.

- [27] B. Dhankour, "Hair Loss Dataset," Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/brijlaldhankour/hair-loss-dataset>. [Accessed: DD Mon. YYYY].
- [28] "Hair Fall Problem Survey Dataset," Mendeley Data, V1, 2023. [Online]. Available: <https://data.mendeley.com/datasets/g46n66frh/1>. [Accessed: DD Mon. YYYY].
- [29] J. Cohen, "A coefficient of agreement for nominal scales," Educational and Psychological Measurement, vol. 20, no. 1, pp. 37–46, 1960.
- [30] J. Cohen, "Weighted kappa: Nominal scale agreement with provision for scaled disagreement or partial credit," Psychological Bulletin, vol. 70, no. 4, pp. 213–220, 1968.
- [31] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," Journal of the American Statistical Association, vol. 22, no. 158, pp. 209–212, 1927.
- [32] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

APPENDIX A

SOURCE CODE

The experiments were run in Python in cloud notebooks (Section 3.8.8). The code is given in the order in which it was run.

A.1 Environment setup

```
import os
os.environ["NUMBA_DISABLE_JIT"] = "1"
os.environ["CUDA_VISIBLE_DEVICES"] = "0"

!pip install -q catboost tabpfn tabfm shap statsmodels openpyxl
```

A.2 Data loading

```
import os
import time
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from itertools import combinations, product

from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.metrics import (
    confusion_matrix, classification_report, accuracy_score,
    precision_score, recall_score, f1_score, roc_auc_score
)

warnings.filterwarnings("ignore")
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

df = pd.read_csv("/kaggle/input/datasets/nirajanshahi/hairstall/data.csv")
print("Shape:", df.shape)
print("Duplicates:", df.duplicated().sum())
print("Missing values:\n", df.isnull().sum()[df.isnull().sum() > 0])
print("\nhairstall distribution:\n", df["hairstall"].value_counts().sort_index())
```

Output: Shape (21606, 23); Duplicates 0; no missing values; hairstall counts 0 = 9,723, 1 = 7,562, 2 = 4,321.

A.3 Preprocessing

```
df_model = df.drop(columns=["id", "full_name"]).copy()

gender_map = {"Female": 0, "F": 0, "Male": 1, "M": 1, "Other": 2, "O": 2}
df_model["gender"] = df_model["gender"].map(gender_map)

class_names = ["Low", "Moderate", "High"]
X = df_model.drop(columns=["hairstall"])
y = df_model["hairstall"]

print("Gender value counts:\n", df_model["gender"].value_counts())
print("\nX shape:", X.shape, "| y shape:", y.shape)
print("\nClass distribution:\n", y.value_counts().sort_index())
```

Output: X shape (21606, 20); gender counts 1 = 12,109, 0 = 9,485, 2 = 12.

A.4 Class distribution plot

```
class_counts = y.value_counts().sort_index()
plt.figure(figsize=(7, 5))
ax = sns.barplot(x=class_names, y=class_counts.values,
                 palette=["#4F94D4", "#E8A33D", "#D45A47"], edgecolor="black")
for i, v in enumerate(class_counts.values):
    ax.text(i, v, f"{v:,}", ha="center", va="bottom", fontweight="bold")
plt.xlabel("Hair Fall Risk Tier")
plt.ylabel("Number of Records")
plt.title("Class Distribution: Low vs Moderate vs High")
plt.tight_layout()
plt.savefig("class_distribution.png", dpi=150)
plt.show()
```

A.5 Train/test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=RANDOM_STATE
)
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

print("Train shape:", X_train.shape, "| Test shape:", X_test.shape)
print("\nTrain class balance:\n", y_train.value_counts(normalize=True).sort_index())
print("\nTest class balance:\n", y_test.value_counts(normalize=True).sort_index())
```

Output: train (17,284, 20), test (4,322, 20); class proportions 0.450 / 0.350 / 0.200 in both.

A.6 CatBoost tuning and training

[CHECK BEFORE SUBMISSION: the grid-search loop below was written from the procedure described in Section 3.8.4. Replace it with the original notebook cell if it differs.

Delete this note afterwards.]

```
from catboost import CatBoostClassifier

param_grid = {"depth": [4, 6, 8], "learning_rate": [0.01, 0.03, 0.1]}
grid_rows = []

for depth, lr in product(param_grid["depth"], param_grid["learning_rate"]):
    fold_scores = []
    for tr_idx, va_idx in skf.split(X_train, y_train):
        fold_model = CatBoostClassifier(
            iterations=1000, depth=depth, learning_rate=lr,
            early_stopping_rounds=50, loss_function="MultiClass",
            eval_metric="TotalF1", random_seed=RANDOM_STATE,
            verbose=0, l2_leaf_reg=3.0
        )
        fold_model.fit(
            X_train.iloc[tr_idx], y_train.iloc[tr_idx],
            eval_set=(X_train.iloc[va_idx], y_train.iloc[va_idx]),
            use_best_model=True
        )
        fold_pred = fold_model.predict(X_train.iloc[va_idx]).flatten()
        fold_scores.append(f1_score(y_train.iloc[va_idx], fold_pred, average="macro"))
    grid_rows.append({"depth": depth, "learning_rate": lr,
                     "cv_macro_f1": np.mean(fold_scores)})

grid_results = (pd.DataFrame(grid_rows)
                .sort_values("cv_macro_f1", ascending=False)
                .reset_index(drop=True))
print(grid_results.to_string(index=False))

best_depth = int(grid_results.loc[0, "depth"])
```

```

best_lr = float(grid_results.loc[0, "learning_rate"])
print(f"\nBest: depth={best_depth}, learning_rate={best_lr}, "
      f"CV macro-F1={grid_results.loc[0, 'cv_macro_f1']:.4f}")

cb_model = CatBoostClassifier(
    iterations=1000, depth=best_depth, learning_rate=best_lr,
    early_stopping_rounds=50, loss_function="MultiClass",
    eval_metric="TotalF1", random_seed=RANDOM_STATE,
    verbose=0, l2_leaf_reg=3.0
)

t0 = time.time()
cb_model.fit(X_train, y_train, eval_set=(X_test, y_test), use_best_model=True)
cb_train_time = time.time() - t0

t0 = time.time()
y_pred_cb = cb_model.predict(X_test).flatten()
cb_predict_time = time.time() - t0
y_proba_cb = cb_model.predict_proba(X_test)
print(f"Training time: {cb_train_time:.2f}s | "
      f"Inference: {(cb_predict_time / len(X_test)) * 1000:.4f} ms/sample")

```

Result: best setting depth = 4, learning rate = 0.1, cross-validation macro-F1 = 0.7665. The final fit uses the test partition as the early-stopping evaluation set (see Section 5.2).

A.7 TabPFN inference

```

import torch
from kaggle_secrets import UserSecretsClient

os.environ["TABPFN_TOKEN"] = UserSecretsClient().get_secret("TABPFN_TOKEN")

# Allow the pretrained checkpoint to load with recent PyTorch versions
_original_torch_load = torch.load
def _patched_torch_load(*args, **kwargs):
    kwargs["weights_only"] = False
    return _original_torch_load(*args, **kwargs)
torch.load = _patched_torch_load

from tabpfn import TabPFNClassifier

tabpfn_model = TabPFNClassifier(device="cuda", random_state=RANDOM_STATE)

t0 = time.time()
tabpfn_model.fit(X_train, y_train)
tabpfn_fit_time = time.time() - t0

t0 = time.time()
y_pred_tabpfn = tabpfn_model.predict(X_test)
tabpfn_predict_time = time.time() - t0
y_proba_tabpfn = tabpfn_model.predict_proba(X_test)
print(f"Fit time: {tabpfn_fit_time:.2f}s | "
      f"Inference: {(tabpfn_predict_time / len(X_test)) * 1000:.4f} ms/sample")

```

A.8 TabFM inference (500-row context)

```

import sys
sys.modules["flax.nnx"] = None # force the PyTorch implementation

from tabfm import TabFMClassifier
from tabfm import tabfm_v1_0_0_pytorch as tabfm_v1_0_0

tabfm_base = tabfm_v1_0_0.load(model_type="classification")

np.random.seed(RANDOM_STATE)
context_idx = np.random.choice(X_train.index, size=500, replace=False)
X_train_context = X_train.loc[context_idx]

```

```

y_train_context = y_train.loc[context_idx]

tabfm_model = TabFMClassifier(model=tabfm_base, n_estimators=1)

t0 = time.time()
tabfm_model.fit(X_train_context, y_train_context)
tabfm_fit_time = time.time() - t0

t0 = time.time()
y_pred_tabfm = tabfm_model.predict(X_test)
tabfm_predict_time = time.time() - t0
y_proba_tabfm = tabfm_model.predict_proba(X_test)
print(f"Fit time: {tabfm_fit_time:.2f}s")
print(f"Predict time: {tabfm_predict_time / 60:.1f} min | "
      f"Per-sample: {(tabfm_predict_time / len(X_test)) * 1000:.2f} ms")

```

A.9 Evaluation

```

def evaluate_model(name, y_true, y_pred, y_proba, cmap, filename):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    metrics = {
        "Model": name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Macro_Precision": precision_score(y_true, y_pred, average="macro"),
        "Macro_Recall": recall_score(y_true, y_pred, average="macro"),
        "Macro_F1": f1_score(y_true, y_pred, average="macro"),
        "ROC_AUC": roc_auc_score(y_true, y_proba, multi_class="ovr", average="macro"),
    }
    print(f"\n{name}")
    print(classification_report(y_true, y_pred, target_names=class_names))
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt="d", cmap=cmap,
                xticklabels=class_names, yticklabels=class_names)
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.title(f"{name} - Confusion Matrix")
    plt.tight_layout()
    plt.savefig(filename, dpi=150)
    plt.show()
    return metrics

results_table = pd.DataFrame([
    evaluate_model("CatBoost", y_test, y_pred_cb, y_proba_cb, "Blues",
                  "catboost_confusion_matrix.png"),
    evaluate_model("TabPFN", y_test, y_pred_tabPFN, y_proba_tabPFN, "Greens",
                  "tabPFN_confusion_matrix.png"),
    evaluate_model("TabFM", y_test, y_pred_tabfm, y_proba_tabfm, "Oranges",
                  "tabfm_confusion_matrix.png"),
])
results_table.insert(1, "Training_Context", [len(X_train), len(X_train),
len(X_train_context)])
print(results_table.to_string(index=False))
results_table.to_csv("final_three_way_results.csv", index=False)

```

A.10 McNemar's test

```

from statsmodels.stats.contingency_tables import mcnemar

y_true_values = y_test.values
correct = {
    "CatBoost": np.asarray(y_pred_cb) == y_true_values,
    "TabPFN": np.asarray(y_pred_tabPFN) == y_true_values,
    "TabFM": np.asarray(y_pred_tabfm) == y_true_values,
}

mcnemar_rows = []

```

```

for name_a, name_b in combinations(correct.keys(), 2):
    a, b = correct[name_a], correct[name_b]
    table = [
        [int(np.sum(a & b)), int(np.sum(a & ~b))],
        [int(np.sum(~a & b)), int(np.sum(~a & ~b))],
    ]
    outcome = mcnemar(table, exact=False, correction=True)
    mcnemar_rows.append({
        "Comparison": f"{name_a} vs {name_b}",
        "McNemar_Statistic": outcome.statistic,
        "p_value": outcome.pvalue,
        "Significant": outcome.pvalue < 0.05,
    })

statistical_results = pd.DataFrame(mcnemar_rows)
print(statistical_results.to_string(index=False))
statistical_results.to_csv("final_statistical_analysis.csv", index=False)

```

A.11 Training context experiment

[CHECK BEFORE SUBMISSION: the original notebook had separate cells for the 100-, 500- and 2,000-row runs. The version below combines them into one procedure. Replace it with the original cells if they differ, and confirm that the 100-row run also drew its rows with seed 42. Delete this note afterwards.]

```

from catboost import CatBoostClassifier
from tabpfnc import TabPFNCClassifier
from tabfm import TabFMClassifier

np.random.seed(RANDOM_STATE)
subset_idx = np.random.choice(X_test.index, size=500, replace=False)
X_test_500 = X_test.loc[subset_idx]
y_test_500 = y_test.loc[subset_idx]

eval_sets = {
    100: (X_test, y_test),
    500: (X_test_500, y_test_500),
    2000: (X_test_500, y_test_500),
}

def sample_context(size):
    np.random.seed(RANDOM_STATE)
    idx = np.random.choice(X_train.index, size=size, replace=False)
    return X_train.loc[idx], y_train.loc[idx]

def run_context(size):
    X_ctx, y_ctx = sample_context(size)
    X_eval, y_eval = eval_sets[size]
    rows = []

    cb = CatBoostClassifier(
        iterations=1000, depth=best_depth, learning_rate=best_lr,
        early_stopping_rounds=50, loss_function="MultiClass",
        eval_metric="TotalF1", random_seed=RANDOM_STATE,
        verbose=0, l2_leaf_reg=3.0
    )
    cb.fit(X_ctx, y_ctx, eval_set=(X_eval, y_eval), use_best_model=True)
    rows.append(("CatBoost", cb.predict(X_eval).flatten()))

    pfn = TabPFNCClassifier(device="cuda", random_state=RANDOM_STATE)
    pfn.fit(X_ctx, y_ctx)
    rows.append(("TabPFN", pfn.predict(X_eval)))

    fm = TabFMClassifier(model=tabfm_base, n_estimators=1)
    fm.fit(X_ctx, y_ctx)
    rows.append(("TabFM", fm.predict(X_eval)))

```

```

    return [
        {"Context_Size": size, "Model": name, "Test_Set_Size": len(y_eval),
         "Accuracy": accuracy_score(y_eval, p),
         "Macro_F1": f1_score(y_eval, p, average="macro")}
        for name, p in rows
    ]

scaling_rows = []
for size in [100, 500, 2000]:
    t0 = time.time()
    scaling_rows.extend(run_context(size))
    print(f"Context {size} done in {(time.time() - t0) / 60:.1f} min")

scaling_results = pd.DataFrame(scaling_rows)
print(scaling_results.to_string(index=False))
scaling_results.to_csv("context_scaling_results.csv", index=False)

plot_df = scaling_results.pivot(index="Context_Size", columns="Model",
                                values="Accuracy").sort_index()
x_labels = [str(i) for i in plot_df.index]
plt.figure(figsize=(8, 5))
for model_name in ["CatBoost", "TabPFN", "TabFM"]:
    plt.plot(x_labels, plot_df[model_name] * 100, marker="o", label=model_name)
plt.xlabel("Training Context Size (rows)")
plt.ylabel("Accuracy (%)")
plt.title("Accuracy vs Training Context Size")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("context_scaling_accuracy.png", dpi=150)
plt.show()

```

A.12 SHAP explanations

[CHECK BEFORE SUBMISSION: the random seeds in the SHAP sampling and the 800-row TabPFN context were reconstructed. The Step 8b output line says "30 test samples, 800-row reduced context", but the original sampling cell had no fixed seed. Replace this block with the original notebook cells if they differ. Delete this note afterwards.]

```

import shap

high_class_idx = 2

# --- CatBoost: exact TreeExplainer on the full test partition ---
explainer_cb = shap.TreeExplainer(cb_model)
shap_values_cb = explainer_cb.shap_values(X_test)

shap.summary_plot(shap_values_cb, X_test, class_names=class_names,
                  plot_type="bar", show=False)
plt.title("CatBoost - Global Feature Importance (SHAP)")
plt.tight_layout()
plt.savefig("catboost_shap_global_importance.png", dpi=150, bbox_inches="tight")
plt.show()

if isinstance(shap_values_cb, list):
    shap_vals_high_cb = shap_values_cb[high_class_idx]
else:
    shap_vals_high_cb = shap_values_cb[:, :, high_class_idx]

if isinstance(explainer_cb.expected_value, (list, np.ndarray)):
    base_value_cb = explainer_cb.expected_value[high_class_idx]
else:
    base_value_cb = explainer_cb.expected_value

def save_beeswarm_and_waterfall(prefix, title, values_high, data, base_value):

```

```

shap.summary_plot(values_high, data, show=False)
plt.title(f"{title} - SHAP Beeswarm (High Risk)")
plt.tight_layout()
plt.savefig(f"{prefix}_shap_beeswarm_high.png", dpi=150, bbox_inches="tight")
plt.show()

explanation = shap.Explanation(
    values=values_high[0], base_values=base_value,
    data=data.iloc[0], feature_names=data.columns.tolist()
)
shap.plots.waterfall(explanation, show=False)
plt.title(f"{title} - Patient Waterfall")
plt.tight_layout()
plt.savefig(f"{prefix}_shap_waterfall_patient.png", dpi=150, bbox_inches="tight")
plt.show()

save_beeswarm_and_waterfall("catboost", "CatBoost", shap_vals_high_cb, X_test,
base_value_cb)

# --- TabPFN: KernelExplainer with a reduced context ---
np.random.seed(RANDOM_STATE)
small_idx = np.random.choice(X_train.index, size=800, replace=False)
X_train_small = X_train.loc[small_idx]
y_train_small = y_train.loc[small_idx]

tabpfm_small = TabPFNCNNClassifier(device="cuda", random_state=RANDOM_STATE)
tabpfm_small.fit(X_train_small, y_train_small)

def tabpfm_predict_proba_small(X):
    return tabpfm_small.predict_proba(pd.DataFrame(X, columns=X_test.columns))

background_small = shap.sample(X_train_small, 15, random_state=RANDOM_STATE)
explainer_tabpfm_kernel = shap.KernelExplainer(tabpfm_predict_proba_small,
background_small)

np.random.seed(RANDOM_STATE)
sample_idx_kernel = np.random.choice(X_test.index, size=30, replace=False)
X_test_sample_kernel = X_test.loc[sample_idx_kernel].reset_index(drop=True)

t0 = time.time()
shap_values_tabpfm = np.array(
    explainer_tabpfm_kernel.shap_values(X_test_sample_kernel, nsamples=100))
print(f"TabPFN SHAP time: {time.time() - t0:.1f}s | shape:
{shap_values_tabpfm.shape}")

shap.summary_plot(shap_values_tabpfm[:, :, high_class_idx], X_test_sample_kernel,
plot_type="bar", show=False)
plt.title("TabPFN - Global Feature Importance (SHAP)")
plt.tight_layout()
plt.savefig("tabpfm_shap_global_importance.png", dpi=150, bbox_inches="tight")
plt.show()

save_beeswarm_and_waterfall(
    "tabpfm", "TabPFN", shap_values_tabpfm[:, :, high_class_idx],
    X_test_sample_kernel, explainer_tabpfm_kernel.expected_value[high_class_idx])

# --- TabFM: KernelExplainer with the 500-row context ---
def tabfm_predict_proba_padded(X):
    X_df = pd.DataFrame(X, columns=X_test.columns)
    if len(X_df) == 1:
        # single-row workaround
        X_padded = pd.concat([X_df, X_df], ignore_index=True)
        return tabfm_model.predict_proba(X_padded)[0:1]
    return tabfm_model.predict_proba(X_df)

background_tabfm = shap.sample(X_train_context, 15, random_state=RANDOM_STATE)
explainer_tabfm_kernel = shap.KernelExplainer(tabfm_predict_proba_padded,
background_tabfm)

np.random.seed(RANDOM_STATE)
sample_idx_tabfm = np.random.choice(X_test.index, size=20, replace=False)
X_test_sample_tabfm = X_test.loc[sample_idx_tabfm].reset_index(drop=True)

```

```

t0 = time.time()
shap_values_tabfm = np.array(
    explainer_tabfm_kernel.shap_values(X_test_sample_tabfm, nsamples=100))
print(f"TabFM SHAP time: {(time.time() - t0) / 60:.1f} min | shape:
{shap_values_tabfm.shape}")

shap.summary_plot(shap_values_tabfm[:, :, high_class_idx], X_test_sample_tabfm,
                  plot_type="bar", show=False)
plt.title("TabFM - Global Feature Importance (SHAP)")
plt.tight_layout()
plt.savefig("tabfm_shap_global_importance.png", dpi=150, bbox_inches="tight")
plt.show()

save_beeswarm_and_waterfall(
    "tabfm", "TabFM", shap_values_tabfm[:, :, high_class_idx],
    X_test_sample_tabfm, explainer_tabfm_kernel.expected_value[high_class_idx])

```

A.13 Numba diagnostic test

```

import numba

@numba.njit
def test_func(x):
    return x + 1

try:
    result = test_func(5)
    print("Numba JIT test: SUCCESS, result =", result)
except Exception as e:
    print("Numba JIT test: FAILED")
    print(e)

```

A.14 Descriptive analysis of the dataset

This code was run after the main experiments, on the same data file, to produce Table 3.3, Table 4.8 and Table 4.9 (Section 3.8.2).

```

import pandas as pd
from scipy.stats import spearmanr

df = pd.read_csv("data.csv")
df["gender"] = df["gender"].map({"Female": 0, "F": 0, "Male": 1, "M": 1, "Other": 2,
                                "O": 2})
X = df.drop(columns=["id", "full_name", "hair_fall"])
y = df["hair_fall"]

continuous = ["age", "total_protein", "calcium", "iron", "vitamin_d", "alt_liver",
              "manganese", "body_water_content", "stress_level", "total_keratine",
              "hair_texture"]
flags = [c for c in X.columns if c not in continuous and c != "gender"]

# Mean, standard deviation, minimum and maximum of each continuous predictor (Table
3.3)
print(df[continuous].describe().loc[["mean", "std", "min", "max"]].round(2).T)

# Mean of each continuous predictor in each risk tier (Table 4.8)
print(df.groupby("hair_fall")[continuous].mean().round(2).T)

# Spearman rank correlation of each predictor with the ordinal risk tier (Table 4.8)
rho = {c: spearmanr(df[c], y, nan_policy="omit")[0] for c in continuous + flags}
for name, value in sorted(rho.items(), key=lambda kv: -abs(kv[1])):
    print(f"{name:28s}{value:+.3f}")

# Proportion of records with each binary flag in each risk tier (Table 4.9)
print(df.groupby("hair_fall")[flags].mean().round(3).T)

```

A.15 Wilson score intervals for accuracy

This code was run after the main experiments to produce Table 4.6.

```
from statsmodels.stats.proportion import proportion_confint

n_test = 4322
accuracies = {"CatBoost": 0.785053, "TabPFN": 0.785747, "TabFM": 0.778806}
for name, acc in accuracies.items():
    low, high = proportion_confint(round(acc * n_test), n_test, alpha=0.05,
    method="wilson")
    print(f"{name}: {acc*100:.2f}% 95% Wilson interval {low*100:.2f}% to {high*100:.2f}%")
```

A.16 Ordinal-aware measures

This code was run after the main experiments, on the confusion matrices of Fig. 4.1 to Fig. 4.3, to produce Table 4.4.

```
import numpy as np

confusion = {
    "CatBoost": [[1672, 271, 2], [288, 1114, 111], [1, 256, 607]],
    "TabPFN": [[1654, 290, 1], [268, 1108, 137], [0, 230, 634]],
    "TabFM": [[1615, 328, 2], [250, 1157, 106], [1, 269, 594]],
}
weights = np.array([[ (i - j) ** 2 / 4 for j in range(3)] for i in range(3)])

for name, cm in confusion.items():
    cm = np.array(cm, dtype=float)
    n = cm.sum()
    p_o = np.trace(cm) / n
    p_e = (cm.sum(0) * cm.sum(1)).sum() / n ** 2
    kappa = (p_o - p_e) / (1 - p_e)
    expected = np.outer(cm.sum(1), cm.sum(0)) / n
    qwk = 1 - (weights * cm).sum() / (weights * expected).sum()
    within_one = 1 - (cm[0, 2] + cm[2, 0]) / n
    mae = sum(abs(i - j) * cm[i, j] for i in range(3) for j in range(3)) / n
    print(f"{name}: kappa={kappa:.4f} QWK={qwk:.4f} within one tier={within_one*100:.2f}% MAE={mae:.4f}")
```

APPENDIX B

ADDITIONAL FIGURES

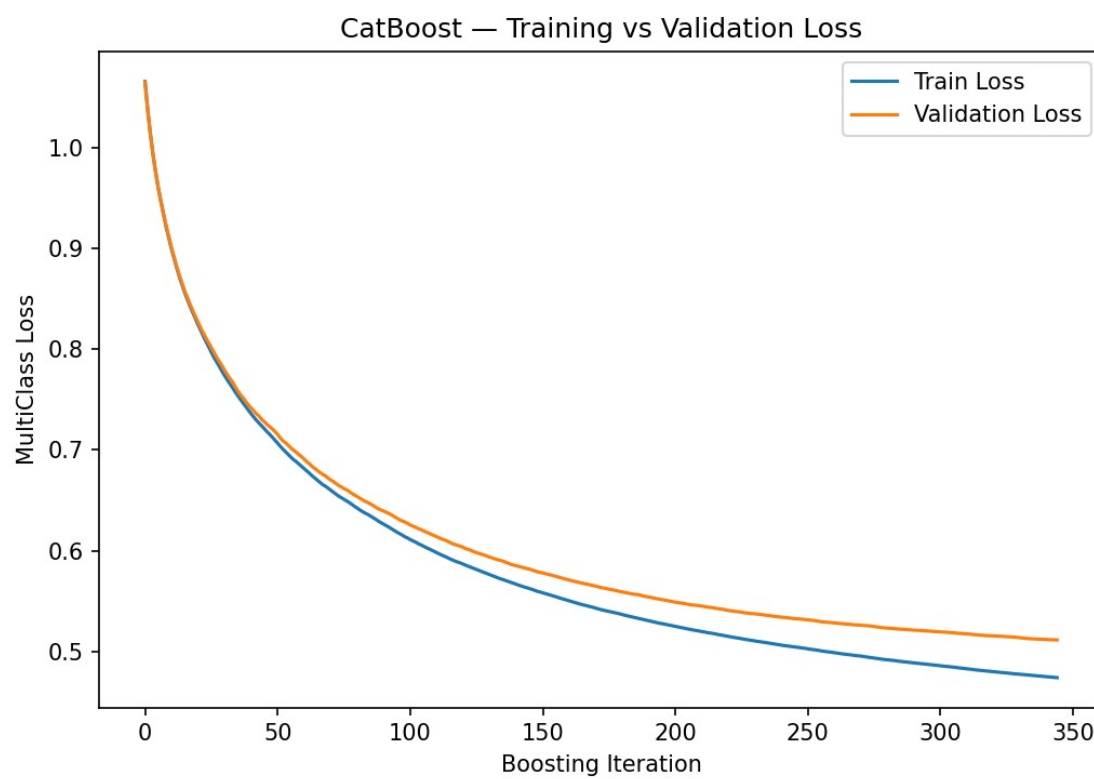


Fig. B.1: CatBoost training curve

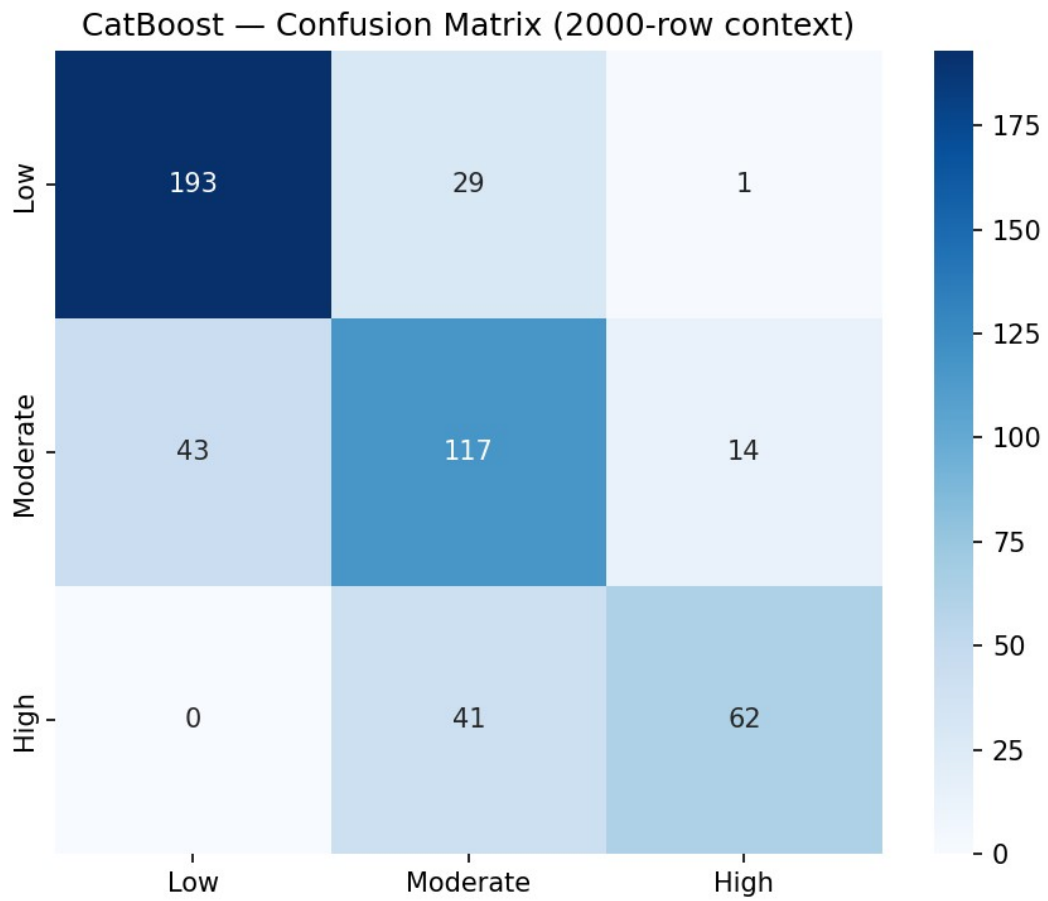


Fig. B.2: CatBoost confusion matrix with a 2,000-row training context (500 test records)

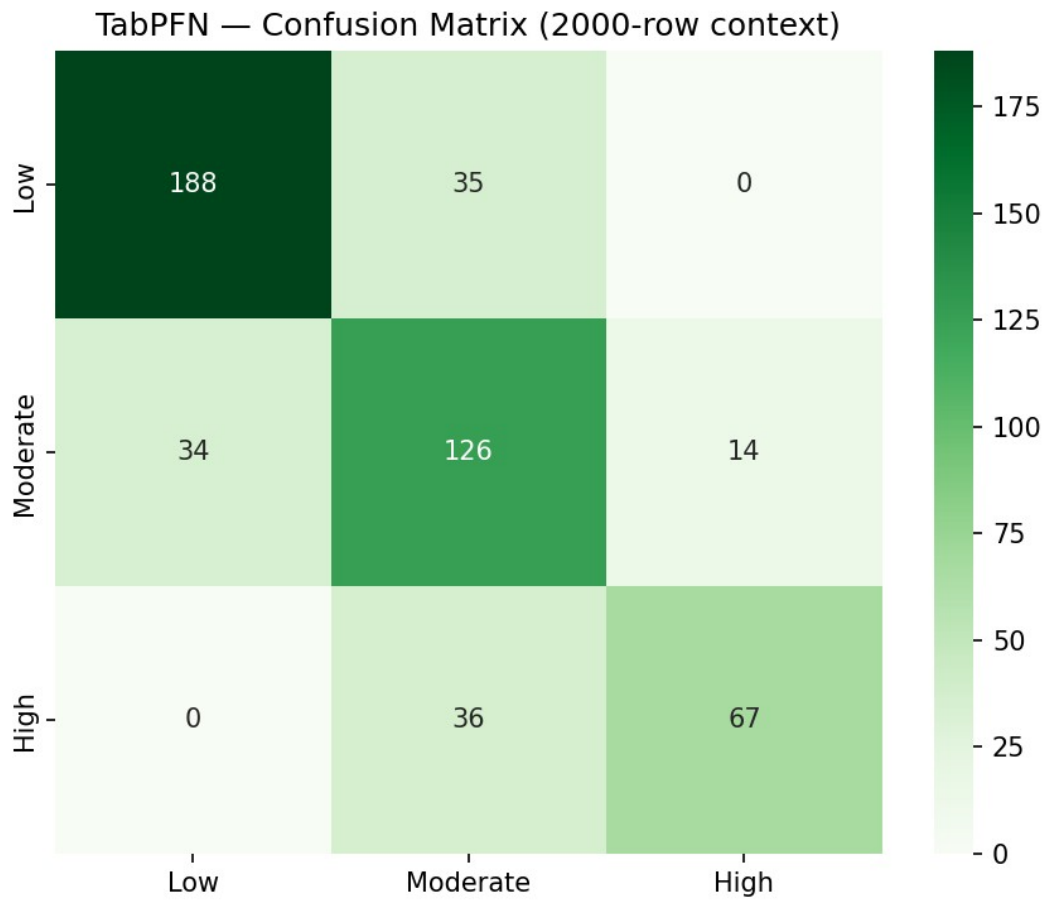


Fig. B.3: TabPFN confusion matrix with a 2,000-row training context (500 test records)

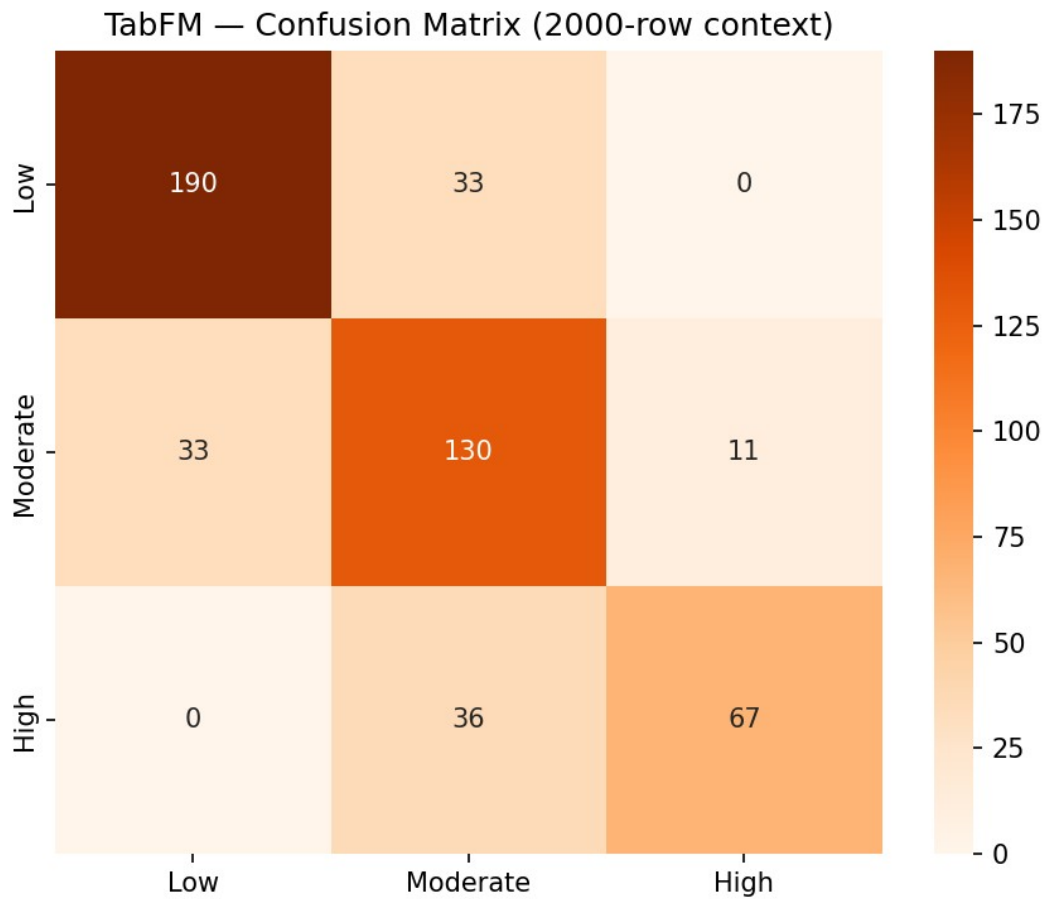


Fig. B.4: TabFM confusion matrix with a 2,000-row training context (500 test records)

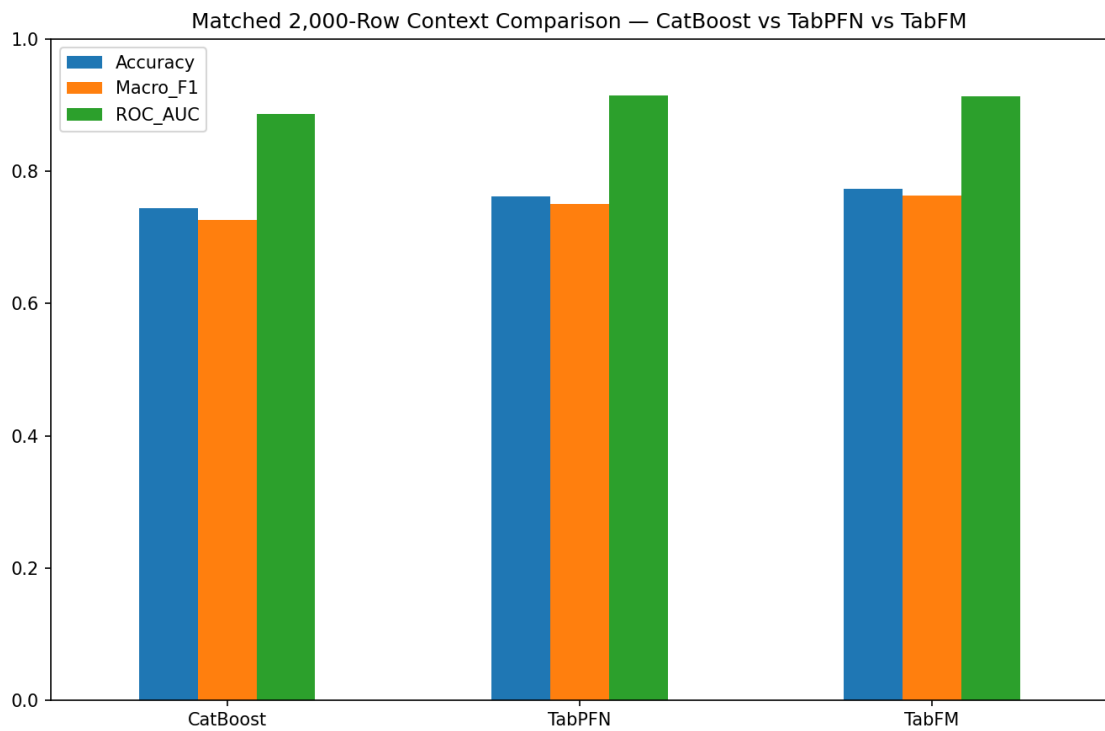


Fig. B.5: Accuracy of the three models with a 2,000-row training context

APPENDIX C

RESULT TABLES FROM THE EXPERIMENTS

Table C.1: Full-context results (final_three_way_results.csv)

Model	Training context	Accuracy	Macro-precision	Macro-recall	Macro-F1	ROC-AUC
CatBoost	17,284	0.7851	0.7915	0.7662	0.7763	0.9189
TabPFN	17,284	0.7857	0.7875	0.7722	0.7787	0.9232
TabFM	500	0.7788	0.7904	0.7608	0.7715	0.9180

Table C.2: McNemar's test (final_statistical_analysis.csv)

Comparison	Statistic	p-value	Significant
CatBoost vs TabPFN	0.0147	0.9037	No
CatBoost vs TabFM	1.6528	0.1986	No
TabPFN vs TabFM	2.5798	0.1082	No

Table C.3: Reduced-context results on the 2,000-row run (matched_2000ctx_results.csv; 500 test records)

Model	Training rows	Test records	Accuracy	Macro-F1	ROC-AUC
CatBoost	2,000	500	0.7440	0.7260	0.8865
TabPFN	2,000	500	0.7620	0.7508	0.9147
TabFM	2,000	500	0.7740	0.7631	0.9139

Table C.4: Per-class precision, recall and F1-score on the test partition

Model	Class	Precision	Recall	F1-score	Support
CatBoost	Low	0.85	0.86	0.86	1,945
CatBoost	Moderate	0.68	0.74	0.71	1,513
CatBoost	High	0.84	0.70	0.77	864
TabPFN	Low	0.86	0.85	0.86	1,945
TabPFN	Moderate	0.68	0.73	0.71	1,513
TabPFN	High	0.82	0.73	0.78	864
TabFM	Low	0.87	0.83	0.85	1,945
TabFM	Moderate	0.66	0.76	0.71	1,513
TabFM	High	0.85	0.69	0.76	864

APPENDIX D

DATASET DESCRIPTION AND SAMPLE RECORDS

The dataset (data.csv) has 21,606 records and 23 columns. The columns id and full_name were removed before modelling, and hair_fall is the target. The 20 predictors are listed in Table 3.2. Table D.1 shows the first three records of the dataset without the name column, with one column per record.

Table D.1: First three records of the dataset (full_name omitted; one column per record)

Column	Record 1	Record 2	Record 3
id	1	2	3
age	39	26	34
gender	Female	Male	Male
total_protein	7.5	7.1	7.6
calcium	9.8	9.5	9.4
iron	56	124	114
vitamin_d	26.8	29.4	23.4
alt_liver	18	29	36
manganese	9.69	10.36	7.74
body_water_content	55.8	46.5	55.4
stress_level	23	31	19
total_keratine	50	18	42
hair_texture	48	76	62
family_hair_fall_history	0	1	1
chronic_illness	0	1	1
late_night_sleep	0	0	0
sleep_disturbance	0	1	1
water_reason	1	0	0
chemical_use	0	0	1
anemia	0	0	0
stress	0	1	0
hair_fall	0	2	1